COLLEGE PLACEMENT MANAGEMENT SYSTEM

Integrated OOPS and DBMS project report

Submitted by

DEVADERSAN R – 250701139

DHARUN KUMAR M – 250701161

DHARUN T M – 250701163

LOHITH HARISH M – 250701391

of

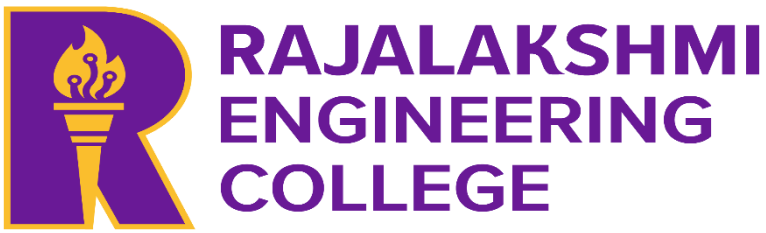
BACHELOR OF ENGINEERING

in

Computer Science and Engineering

RAJALAKSHMI ENGINEERING COLLEGE, THANDALAM

(An Autonomous Institution)



November 2026

RAJALAKSHMI ENGINEERING COLLEGE

BONAFIDE CERTIFICATE

Certified that this project report titled "**COLLEGE PLACEMENT MANAGEMENT SYSTEM**" is the bonafide work of "**DEVADERSAN R (250701139), DHARUN KUMAR M (250701161), DHARUN T M (250701163), LOHITH HARISH M (250701391)**" who carried out the project work under my supervision.

SIGNATURE

Internal Faculty Name

Course Incharge

Object oriented programming system

Department name

SIGNATURE

Internal Faculty Name

Course Incharge

Database management system

Department name

INTERNAL EXAMINER

EXTERNAL EXAMINER

ABSTRACT

The College Placement Management System (CPMS) is a web-based application designed to organize and support the college placement process. It provides separate access for students and placement officers through role-based authentication. The system maintains student academic profiles, company details, job postings, resumes, applications, interview schedules and placement outcomes in a MySQL database. An automatic eligibility checker compares student academic details with the criteria specified for each job. Students can view eligible opportunities, manage applications and track interview updates. Placement officers can manage companies and jobs, monitor applications, schedule interview rounds and communicate through an in-app notification center. The system also includes placement analytics and PDF/Excel report generation. Java Servlets, JSP and JDBC are used for the web application and database access, with a Model–View–Controller structure and DAO classes to organize the code. The project aims to reduce manual work, improve access to placement information and maintain consistent placement records.

TABLE OF CONTENTS

CHAPTER	CONTENTS	PAGE NO.
1	INTRODUCTION	
	1.1 PROBLEM STATEMENT	5
	1.2 PROJECT OVERVIEW	5
	1.3 PURPOSE	5
	1.4 SCOPE	5
2	FUNCTIONALITIES PROVIDED	7
3	SYSTEM FLOW DIAGRAM	8
	3.1 DATAFLOW DIAGRAM	8
	3.2 USE CASE DIAGRAM	9
	3.3 ENTITY RELATIONSHIP DIAGRAM	10
4	MODULE DESCRIPTION	11
5	IMPLEMENTATION	12
	5.1 LANDING PAGE	13
	5.2 STUDENT AND OFFICER LOGIN PAGE	18
	5.3 STUDENT, COMPANY AND JOB ENTRY PAGE	21
	5.4 DASHBOARDS AND RECORDS VIEW PAGE	27
	5.5 UPDATE STUDENT AND JOB RECORDS PAGE	31
	5.6 CLOSE OR DEACTIVATE RECORDS PAGE	37
6	CONCLUSION	41
7	REFERENCES	42

1 - INTRODUCTION

1.1 PROBLEM STATEMENT

Colleges often manage placement activities using separate records and manual communication. This can make it difficult to maintain current student profiles, publish job opportunities, verify eligibility, track applications and coordinate interview schedules. CPMS is proposed to centralize these activities in one web-based system with a shared MySQL database.

1.2 PROJECT OVERVIEW

The College Placement Management System is a web application for students and placement officers. Students can maintain their profiles and resumes, view jobs for which they meet the stated criteria, submit applications and follow application and interview status. Placement officers can maintain company and job information, define eligibility requirements, review applications, arrange interview rounds and send in-app notifications. The system stores information in related MySQL tables and accesses it from Java through JDBC DAO classes. JSP pages present the interface, while Servlets handle requests and coordinate application operations.

1.3 PURPOSE

The purpose of CPMS is to provide a centralized and organized way to manage college placement activities. It is intended to make student and company information easier to maintain, support consistent eligibility checking, simplify application tracking and interview scheduling, and provide timely in-app updates. Placement analytics and PDF/Excel reports are intended to help placement officers review placement activity. Role-based access and controlled database

operations are included in the design to protect records and maintain data consistency.

1.4 SCOPE

The scope includes student registration and profile management, academic records, resume management, company records, job postings, department-wise job criteria, automatic eligibility checking, job applications and application-status tracking. It also includes interview scheduling for multiple rounds, an in-app notification center, placement analytics and PDF/Excel report generation. The application is planned as a web system using Java Servlets, JSP, JDBC, MySQL and Apache Tomcat. Email/SMS delivery, payment processing and external recruitment-platform integration are not part of the agreed scope.

2 FUNCTIONALITIES PROVIDED 7

FUNCTIONALITIES PROVIDED 7

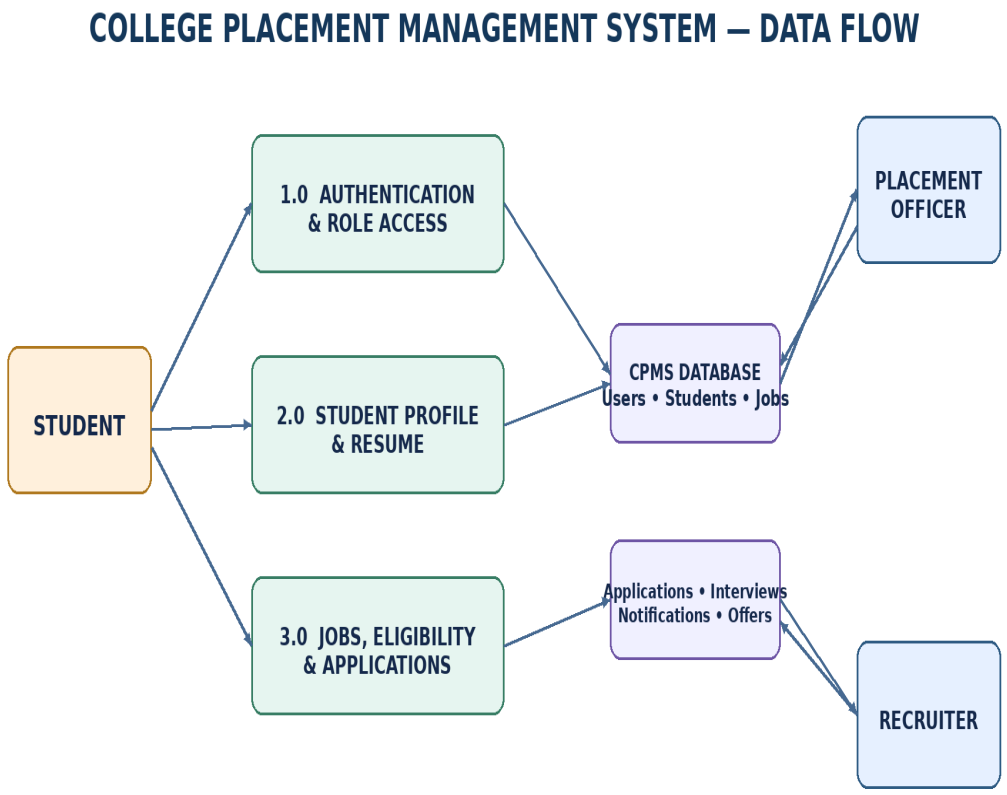
- Role-based registration and login for students and placement officers.
- Student profile, academic record and resume management.
- Company information and job-posting management.
- Automatic checking of student eligibility against job criteria.
- Job search, application submission and application-status tracking.
- Interview scheduling with support for multiple rounds.
- In-app notifications, placement analytics and PDF/Excel report generation.

3 SYSTEM FLOW DIAGRAM 8

The system flow describes how students and placement officers use CPMS. Students register or log in, maintain their profile and resume, view eligible jobs, submit applications and check application and interview updates. Placement officers log in to manage students, companies and job postings, review eligibility and applications, schedule interviews, update outcomes and generate reports. The application uses Servlets and service/DAO classes to communicate with the MySQL database through JDBC.

3.1 DATAFLOW DIAGRAM8

The Data Flow Diagram should show the Student and Placement Officer as external entities, the CPMS application processes, and the MySQL data stores. Major data flows include student profile and resume details, company and job details, eligibility results, applications, interview schedules, notifications and report requests. The diagram in the final submission should be updated to represent these CPMS processes and data stores.



9

The main actors are Student and Placement Officer. Student use cases include register/login, manage profile, upload or manage resume, view eligible jobs, apply for a job, track application status, view interview schedules and read notifications. Placement Officer use cases include login, manage students, departments, companies and jobs, define eligibility criteria, review applications, schedule interview rounds, update interview outcomes, send notifications, view analytics and generate PDF/Excel reports.

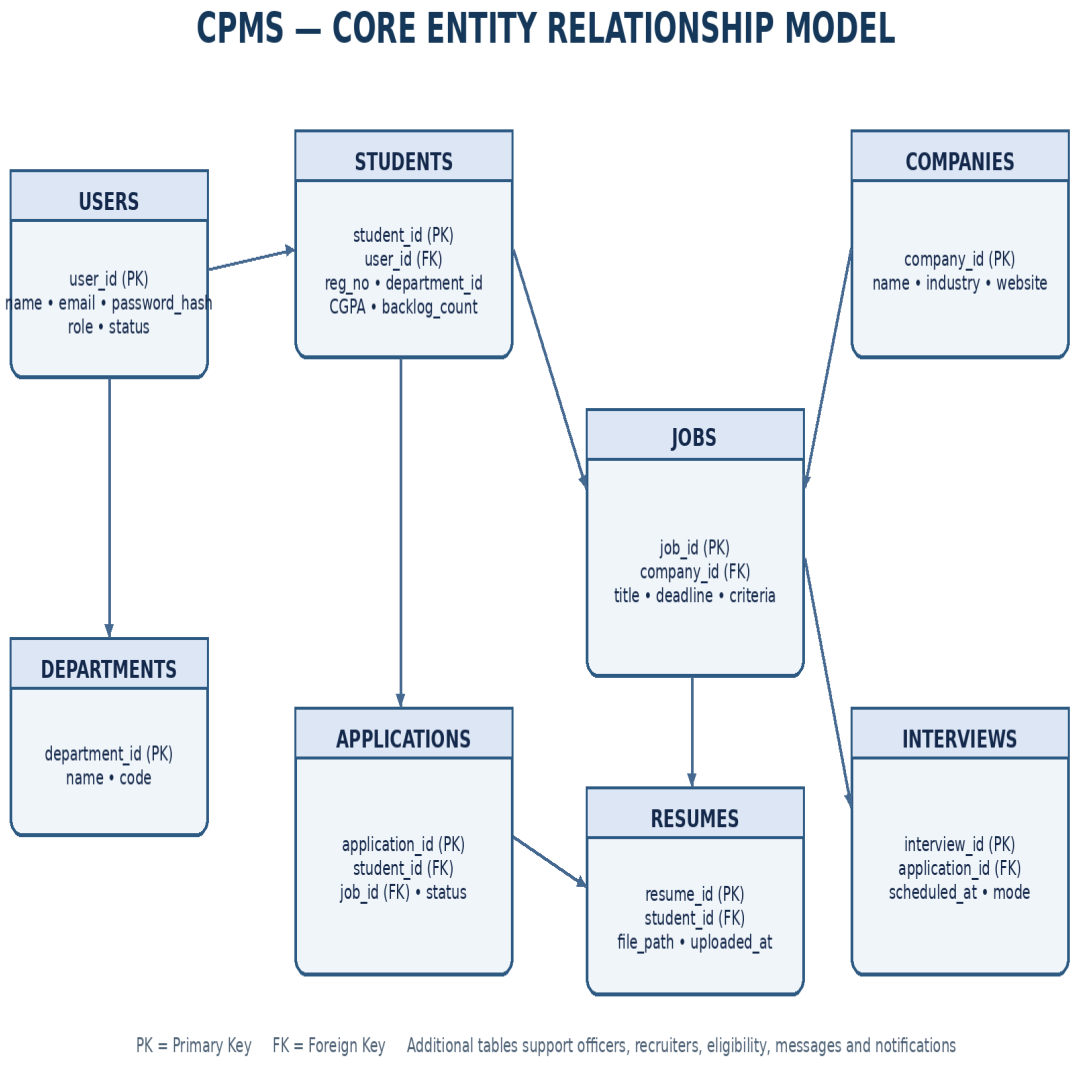
Actor: Student

Use Cases

a. Register Student - Description: Allows an student to add new student details to the system.
b. Update Student - Description: Permits an student to remove an existing student from the system.
c. View Student Profile - Description: Enables an student to view details of other students.
d. Exit System - Description: Allows an student to log out or exit the system.

3.3 ENTITY RELATIONSHIP DIAGRAM 10

The CPMS database is designed around users, students, placement officers, departments, companies, jobs, job_departments, resumes, eligibility_checks, applications, interviews, offers, notifications and report_history. The users table stores login identity and role. Student and placement-officer records extend user information. Companies have job postings; jobs can be associated with multiple departments through job_departments. Students can have resumes and eligibility checks, and can submit applications to jobs. Applications can have interview rounds and an offer record. Users can receive notifications, while report_history records generated report metadata. Primary keys, foreign keys, uniqueness rules and indexes support referential integrity and common lookups.



4 MODULE DESCRIPTION 11

The project is organized into the following functional modules:

AUTHENTICATION MODULE:

Provides registration and login for students and placement officers. The user role determines which pages and operations are available after login.

STUDENT AND RESUME MANAGEMENT MODULE:

Maintains student profile information, department, academic details, contact information and resume records. Students can update their permitted profile fields and manage their resume.

COMPANY AND JOB MANAGEMENT MODULE:

Allows placement officers to maintain company information and create job postings with role descriptions, minimum CGPA, arrears limit, eligible graduation year, package and application deadline.

ELIGIBILITY AND APPLICATION MODULE:

Checks student academic details against job criteria and presents eligible opportunities. Students can submit applications, while placement officers review applications and update their status.

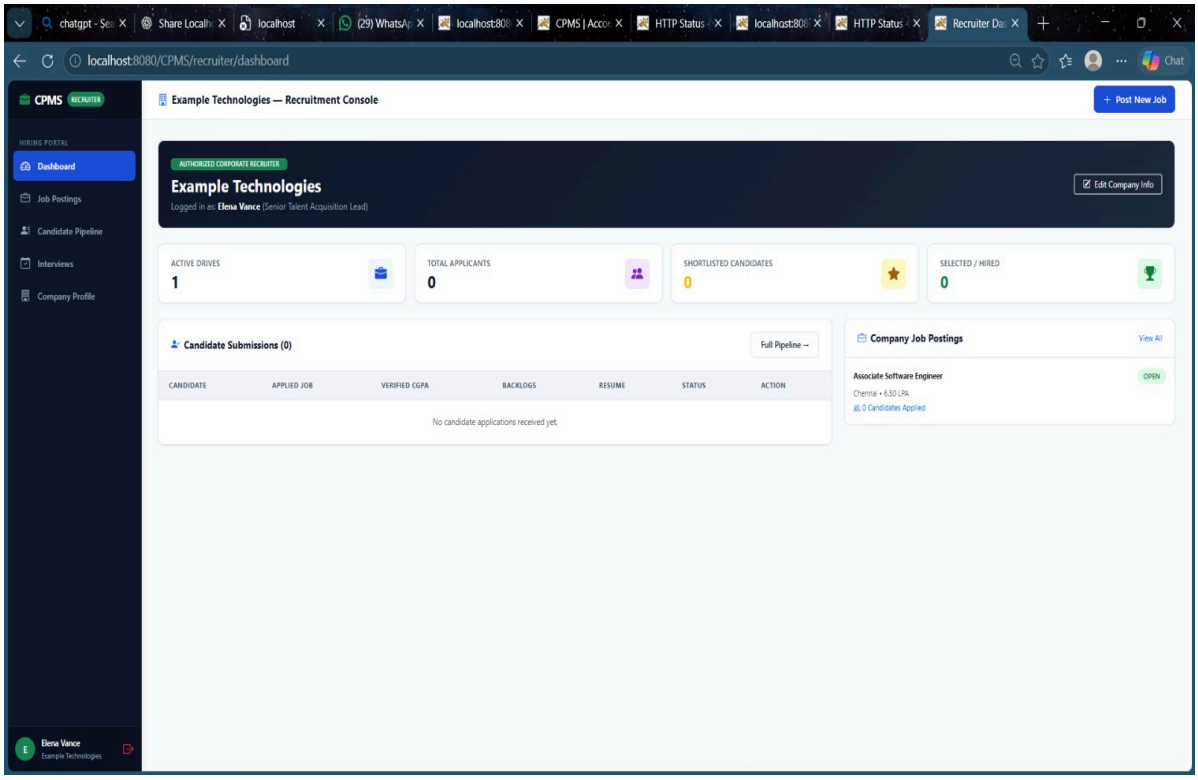
INTERVIEW SCHEDULING MODULE:

Supports scheduling multiple interview rounds for an application, including date and time, interview mode, venue or meeting link, and round result.

5 IMPLEMENTATION 12

5.1 NOTIFICATION CENTER:

Stores in-app notifications for job announcements, application updates, interview reminders and selection updates. Users can view notifications and mark them as read.



ANALYTICS AND REPORT GENERATION:

Provides placement and department-wise summaries. Placement officers can request PDF and Excel reports, with generated report details recorded in report_history.

APPLICATION ARCHITECTURE:

IMPLEMENTATION 12
5.1 LANDING PAGE:

webapp/index.jsp

```
<%@ page contentType="text/html;charset=UTF-8" language="java" %>
<%@ taglib prefix="c" uri="jakarta.tags.core" %>
<%
  // Auto-redirect authenticated users directly to their respective dashboard
  if (session != null && session.getAttribute("role") != null) {
    String userRole = (String) session.getAttribute("role");
    if ("STUDENT".equalsIgnoreCase(userRole)) {
      response.sendRedirect(request.getContextPath() + "/student/dashboard");
      return;
    } else if ("OFFICER".equalsIgnoreCase(userRole)) {
      response.sendRedirect(request.getContextPath() + "/officer/dashboard");
      return;
    } else if ("RECRUITER".equalsIgnoreCase(userRole)) {
      response.sendRedirect(request.getContextPath() + "/recruiter/dashboard");
      return;
    }
  }
%>
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>College Placement Management System (CPMS)</title>
  <!-- Bootstrap 5 CSS -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
  <!-- Bootstrap Icons -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-icons.min.css" rel="stylesheet">
  <!-- CPMS Theme -->
  <link href="${pageContext.request.contextPath}/assets/css/cpms-theme.css" rel="stylesheet">
  <style>
    .hero-section {
      background: linear-gradient(135deg, #0f172a 0%, #1e293b 100%);
      color: #ffffff;
      padding: 4rem 0 5rem;
      position: relative;
      overflow: hidden;
    }
    .hero-section::before {
      content: '';
      position: absolute;
      top: 0; left: 0; right: 0; bottom: 0;
      background: radial-gradient(circle at 80% 20%, rgba(37, 99, 235, 0.15), transparent 40%);
      pointer-events: none;
    }
    .role-card {
      background: #ffffff;
      border: 1px solid var(--cpms-slate-200);
      border-radius: 16px;
      padding: 2rem 1.75rem;
      transition: all 0.3s ease;
      height: 100%;
      display: flex;
      flex-direction: column;
      box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.05);
    }
    .role-card:hover {
      transform: translateY(-6px);
      box-shadow: 0 15px 30px rgba(0, 0, 0, 0.08);
      border-color: var(--cpms-blue-400);
```

MYSQL DATABASE CONNECTION USING JDBC:

The DBConnection utility is responsible

for obtaining a JDBC connection to the

MySQL CPMS database. Database URL,

username and password are supplied through environment configuration rather than being written directly in source code. DAO classes use PreparedStatement for database operations and close resources after use. Multi-step operations, such as application and notification updates, should use JDBC transactions.

util/DBConnection.java

```
package com.cpms.util;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DBConnection {

    private static final String URL =
        System.getenv().getOrDefault(
            "CPMS_DB_URL",
            "jdbc:mysql://localhost:3306/cpms?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=Asia/Kolkata"
        );

    private static final String USER =
        System.getenv().getOrDefault("CPMS_DB_USER", "root");

    private static final String PASSWORD =
        System.getenv().getOrDefault("CPMS_DB_PASSWORD", "");

    public static Connection getConnection() throws SQLException {

        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
        } catch (ClassNotFoundException e) {
            throw new SQLException(
                "MySQL JDBC Driver could not be loaded.", e
            );
        }

        return DriverManager.getConnection(URL, USER, PASSWORD);
    }
}
```

5.2 Student workspace

The student workspace brings the main placement tasks together in one consistent interface. After signing in, a student can review profile completeness, maintain academic information, upload or replace a resume, and browse opportunities published by the placement office. The dashboard presents relevant status information so that the student can understand which actions are pending and which applications have progressed. Separating profile, resume and job views keeps each task focused while the common navigation makes movement between modules straightforward.

JAVA PACKAGE AND APPLICATION STRUCTURE:

The Java source is organized into

model, controller, service, dao, report,

util and filter packages. Model classes

represent users, students, companies,

jobs, applications, interviews and

notifications. Servlets handle web

requests; services apply rules; DAO

classes perform SQL operations.

AuthenticationFilter protects pages that

require a signed-in user.

Package structure

```
com.cpms
  model/      User, Student, PlacementOfficer, Recruiter, Company, Job, Application,
              Interview, Resume, ResumeFeedback, Notification, Message,
              AcademicUpdate, EligibilityResult, Department
  controller/  LoginServlet, RegisterServlet, LogoutServlet, Student*Servlet,
              Officer*Servlet, Recruiter*Servlet, ResumeDownloadServlet
  service/     AuthService, StudentService, OfficerService, RecruiterService,
              EligibilityService, NotificationService, ReportService
  dao/        UserDAO, StudentDAO, CompanyDAO, JobDAO, ApplicationDAO, InterviewDAO,
              ResumeDAO, NotificationDAO, MessageDAO, ReportDAO, DepartmentDAO, ...
  report/     PDF and Excel report generation classes
  filter/     AuthFilter
  util/       DBConnection, PasswordUtil
  test/       DatabaseConnectionTest, PasswordVerificationTest
webapp/       index.jsp, login.jsp, register.jsp, student/, officer/, recruiter/,
              assets/css/cpms-theme.css, WEB-INF/web.xml
```

webapp/WEB-INF/web.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
        https://jakarta.ee/xml/ns/jakartaee/web-app_6_0.xsd"
        version="6.0">

    <display-name>College Placement Management System</display-name>

    <filter>
        <filter-name>AuthFilter</filter-name>
        <filter-class>com.cpms.filter.AuthFilter</filter-class>
    </filter>

    <filter-mapping>
        <filter-name>AuthFilter</filter-name>
        <url-pattern>/student/*</url-pattern>
    </filter-mapping>

    <filter-mapping>
        <filter-name>AuthFilter</filter-name>
        <url-pattern>/officer/*</url-pattern>
    </filter-mapping>
```

```
    <filter-mapping>
        <filter-name>AuthFilter</filter-name>
        <url-pattern>/admin/*</url-pattern>
    </filter-mapping>

    <filter-mapping>
        <filter-name>AuthFilter</filter-name>
        <url-pattern>/recruiter/*</url-pattern>
    </filter-mapping>

    <filter-mapping>
        <filter-name>AuthFilter</filter-name>
        <url-pattern>/resume/*</url-pattern>
    </filter-mapping>

    <session-config>
        <session-timeout>30</session-timeout>
        <cookie-config>
            <http-only>true</http-only>
            <secure>false</secure>
        </cookie-config>
        <tracking-mode>COOKIE</tracking-mode>
    </session-config>

    <welcome-file-list>
        <welcome-file>login.jsp</welcome-file>
    </welcome-file-list>

</web-app>
```

5.3 Authentication and role-based navigation

The login workflow validates the supplied credentials against the application database and establishes a session for the authenticated user. The application uses the role stored for the account to direct the user to the appropriate workspace. Student, Placement Officer and Recruiter functions are kept separate so that each role sees only the operations relevant to its responsibilities. Protected routes are checked by the authentication filter before a page is served. This approach provides a clear boundary between public pages and role-specific functions.

5.2 STUDENT AND PLACEMENT OFFICER LOGIN PAGE18

The login screen is the common entry point for registered users. The user provides the registered email address and password, and the application validates the credentials before creating an authenticated session. The role associated with the account determines the destination workspace. Students are directed to student-facing functions such as profile maintenance, resume management and job applications. Placement officers are directed to administrative functions such as student records, company details and job postings. The same authentication process therefore supports multiple user groups without mixing their permissions. A session-based workflow also allows protected pages to verify that a user has signed in before displaying private placement information. Invalid credentials are rejected and the user can retry without gaining access to protected modules.

STUDENT AND PLACEMENT OFFICER LOGIN PAGE 18

The login screen is the common entry point for registered users. The user provides the registered email address and password, and the application validates the credentials before creating an authenticated session. The role associated with the account determines the destination workspace. Students are directed to student-facing functions such as profile maintenance, resume management and job applications. Placement officers are directed to administrative functions such as student records, company details and job postings. The same authentication process therefore supports multiple user groups without mixing their permissions. A session-based workflow also allows protected pages to verify that a user has signed in before displaying private placement information. Invalid credentials are rejected and the user can retry without gaining access to protected modules.

controller/LoginServlet.java

```
package com.cpms.controller;

@WebServlet("/login")
public class LoginServlet extends HttpServlet {

   private AuthService authService;
   private StudentDAO studentDAO;
   private PlacementOfficerDAO officerDAO;
   private RecruiterDAO recruiterDAO;

   @Override
   public void init() {
      authService = new AuthService();
      studentDAO = new StudentDAO();
      officerDAO = new PlacementOfficerDAO();
      recruiterDAO = new RecruiterDAO();
   }

   @Override
   protected void doGet(HttpServletRequest request, HttpServletResponse response)
         throws ServletException, IOException {
      HttpSession session = request.getSession(false);
      if (session != null && session.getAttribute("role") != null) {
         String role = (String) session.getAttribute("role");
         if ("STUDENT".equalsIgnoreCase(role)) {
            response.sendRedirect(request.getContextPath() + "/student/dashboard");
            return;
         } else if ("OFFICER".equalsIgnoreCase(role)) {
            response.sendRedirect(request.getContextPath() + "/officer/dashboard");
            return;
         } else if ("RECRUITER".equalsIgnoreCase(role)) {
            response.sendRedirect(request.getContextPath() + "/recruiter/dashboard");
            return;
         }
      }
      request.getRequestDispatcher("/login.jsp").forward(request, response);
   }

   @Override
   protected void doPost(HttpServletRequest request,
                         HttpServletResponse response)
         throws ServletException, IOException {

      String email = request.getParameter("email");
      String password = request.getParameter("password");

      User user = authService.login(email, password);

      if (user == null) {
         request.setAttribute("errorMessage", "Invalid email or password.");
```

```
            request.getRequestDispatcher("/login.jsp").forward(request, response);
            return;
        }

        // Create a new session after successful authentication
        HttpSession session = request.getSession(true);

        // Prevent session fixation
        request.changeSessionId();

        session.setAttribute("userId", user.getUserId());
        session.setAttribute("fullName", user.getFullName());
        session.setAttribute("email", user.getEmail());
        session.setAttribute("role", user.getRole());

        // Cache role-specific identifiers in session
        if ("STUDENT".equalsIgnoreCase(user.getRole())) {
            Student s = studentDAO.findByUserId(user.getUserId());
            if (s != null) {
                session.setAttribute("studentId", s.getStudentId());
                session.setAttribute("registerNumber", s.getRegisterNumber());
            }
            session.setMaxInactiveInterval(30 * 60);
            response.sendRedirect(request.getContextPath() + "/student/dashboard");

        } else if ("OFFICER".equalsIgnoreCase(user.getRole())) {
            PlacementOfficer po = officerDAO.findByUserId(user.getUserId());
            if (po != null) {
                session.setAttribute("officerId", po.getOfficerId());
            }
            session.setMaxInactiveInterval(30 * 60);
            response.sendRedirect(request.getContextPath() + "/officer/dashboard");

        } else if ("RECRUITER".equalsIgnoreCase(user.getRole())) {
            Recruiter r = recruiterDAO.findByUserId(user.getUserId());
            if (r != null) {
                session.setAttribute("recruiterId", r.getRecruiterId());
                session.setAttribute("companyId", r.getCompanyId());
                session.setAttribute("companyName", r.getCompanyName());
            }
            session.setMaxInactiveInterval(30 * 60);
            response.sendRedirect(request.getContextPath() + "/recruiter/dashboard");

        } else {
            session.invalidate();
            response.sendError(HttpServletResponse.SC_FORBIDDEN, "Unauthorized role.");
        }
    }
}
```

webapp/login.jsp

```
    <form action="${pageContext.request.contextPath}/login" method="post">
        <div class="mb-3">
            <label class="form-label small fw-semibold text-muted">Email Address</label>
            <div class="input-group">
                <span class="input-group-text bg-light border-end-0 text-muted"><i class="bi bi-envelope"></i></span>
                <input type="email" id="emailInput" name="email" class="form-control border-start-0" placeholder="name@college.edu" required
autofocus>
            </div>
        </div>

        <div class="mb-4">
            <label class="form-label small fw-semibold text-muted">Password</label>
            <div class="input-group">
                <span class="input-group-text bg-light border-end-0 text-muted"><i class="bi bi-lock"></i></span>
                <input type="password" id="passwordInput" name="password" class="form-control border-start-0" placeholder="••••••••"
required>
            </div>
        </div>

        <button type="submit" class="btn btn-primary-cpms w-100 py-2.5 fw-semibold mb-3">
            Sign In <i class="bi bi-arrow-right ms-1"></i>
        </button>
    </form>

    <!-- Quick Demo Accounts Helper -->
    <div class="border-top pt-3 mt-2">
```

```
        <div class="d-flex align-items-center justify-content-between mb-2">
          <span class="small text-muted fw-semibold"><i class="bi bi-lightning-charge-fill text-warning me-1"></i>Quick Demo
Autofill:</span>
        </div>
        <div class="d-flex flex-wrap gap-2">
          <span class="demo-chip" onclick="fillDemo('student@cpms.test', 'Student@123')">
            <i class="bi bi-person text-primary me-1"></i> Student
          </span>
          <span class="demo-chip" onclick="fillDemo('officer@cpms.test', 'Officer@123')">
            <i class="bi bi-shield-check text-warning me-1"></i> Officer
          </span>
          <span class="demo-chip" onclick="fillDemo('recruiter@example.com', 'Recruiter@123')">
```

5.4 Student profile and academic information

The student profile module maintains the information required during placement activities, including registration details, department and academic performance. The academic record is important because job opportunities may define minimum CGPA and backlog conditions. Students can review their stored information and submit permitted updates, while the placement office can use the same centralized records for verification and placement coordination. Keeping profile data in structured database fields makes it easier to validate and retrieve consistently.

controller/StudentProfileServlet.java

```
package com.cpms.controller;

@WebServlet("/student/profile")
public class StudentProfileServlet extends HttpServlet {

   private StudentService studentService;

   @Override
   public void init() {
      studentService = new StudentService();
   }

   @Override
   protected void doGet(HttpServletRequest request,
                        HttpServletResponse response)
         throws ServletException, IOException {

      HttpSession session = request.getSession(false);
      if (session == null || session.getAttribute("userId") == null) {
         response.sendRedirect(request.getContextPath() + "/login.jsp");
         return;
      }

      int userId = (Integer) session.getAttribute("userId");
      Student student = studentService.getStudentProfile(userId);

      if (student == null) {
         request.setAttribute("errorMessage", "Student profile not found.");
      }

      request.setAttribute("student", student);
      request.getRequestDispatcher("/student/profile.jsp").forward(request, response);
   }

   @Override
   protected void doPost(HttpServletRequest request,
                         HttpServletResponse response)
         throws ServletException, IOException {

      HttpSession session = request.getSession(false);
      if (session == null || session.getAttribute("userId") == null) {
         response.sendRedirect(request.getContextPath() + "/login.jsp");
         return;
      }

      int userId = (Integer) session.getAttribute("userId");

      String phone = request.getParameter("phone");
      String dobValue = request.getParameter("dateOfBirth");
      String address = request.getParameter("address");

      LocalDate dateOfBirth = null;
      try {
         if (dobValue != null && !dobValue.isBlank()) {
            dateOfBirth = LocalDate.parse(dobValue);
         }
      } catch (DateTimeParseException e) {
         request.setAttribute("errorMessage", "Invalid date format. Use YYYY-MM-DD.");
         doGet(request, response);
         return;
      }

      boolean updated = studentService.updateContactDetails(userId, phone, dateOfBirth, address);

      if (updated) {
         response.sendRedirect(request.getContextPath() + "/student/profile?success=true");
      } else {
         request.setAttribute("errorMessage", "Profile update failed. Ensure phone number contains 10-15 digits.");
         doGet(request, response);
      }
   }
}
```

5.5 Resume management

Resume management allows a student to upload a current resume and maintain the document associated with the student account. The application stores the file reference and related metadata so that recruiters and placement staff can access the appropriate candidate document through the system. The interface makes the resume action distinct from academic profile editing, reducing confusion between personal details and supporting documents. A current resume helps ensure that the information considered during screening reflects the student's latest submission.

controller/StudentResumeServlet.java

```
package com.cpms.controller;

@WebServlet("/student/resume")
@MultipartConfig(
    fileSizeThreshold = 1024 * 1024,     // 1 MB
    maxFileSize = 5 * 1024 * 1024,       // 5 MB
    maxRequestSize = 10 * 1024 * 1024    // 10 MB
)
public class StudentResumeServlet extends HttpServlet {

  private StudentService studentService;

  @Override
  public void init() {
    studentService = new StudentService();
  }

  @Override
  protected void doGet(HttpServletRequest request, HttpServletResponse response)
      throws ServletException, IOException {

    HttpSession session = request.getSession(false);
    if (session == null || session.getAttribute("userId") == null) {
      response.sendRedirect(request.getContextPath() + "/login.jsp");
      return;
    }

    int userId = (Integer) session.getAttribute("userId");
    Student student = studentService.getStudentProfile(userId);
    if (student == null) {
      response.sendRedirect(request.getContextPath() + "/student/dashboard");
      return;
    }

    Resume activeResume = studentService.getActiveResume(student.getStudentId());
    List<ResumeFeedback> feedbacks = studentService.getResumeFeedbacks(student.getStudentId());

    request.setAttribute("student", student);
    request.setAttribute("activeResume", activeResume);
    request.setAttribute("feedbacks", feedbacks);
    request.getRequestDispatcher("/student/resume.jsp").forward(request, response);
  }

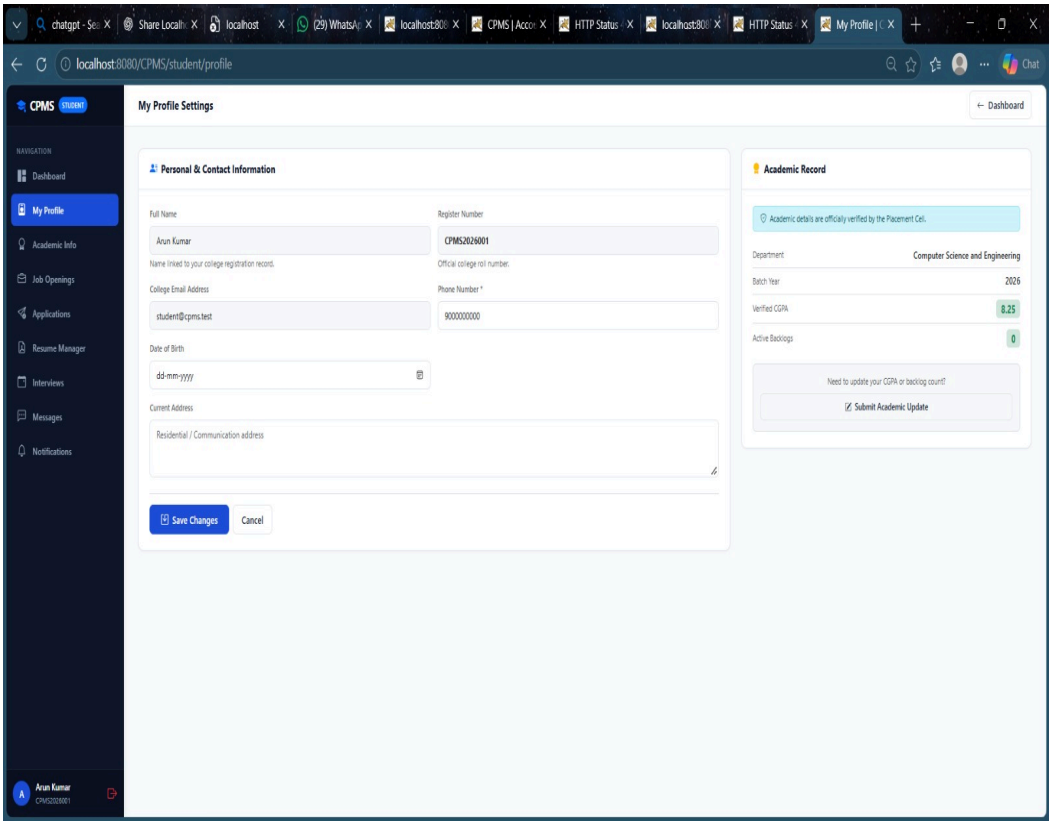
  @Override
  protected void doPost(HttpServletRequest request, HttpServletResponse response)
      throws ServletException, IOException {
```

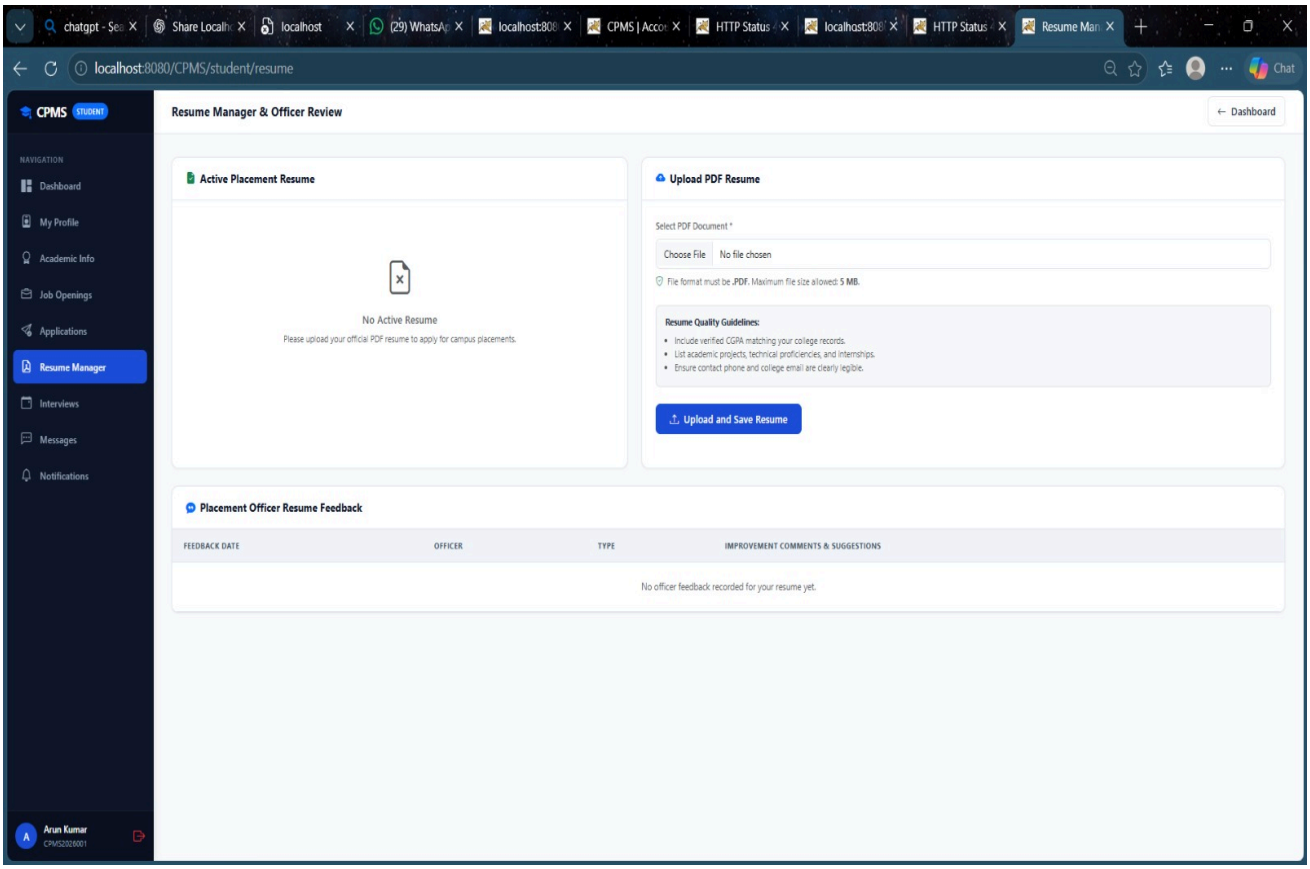
5.3 STUDENT, COMPANY AND JOB ENTRY PAGE:

The entry forms capture the core records needed to operate the placement workflow. Student details are associated with the registered user account and department, while company records describe the organization offering opportunities. Job entries are associated with a company and include the role description, application deadline and eligibility conditions. Keeping these records separate avoids duplicating company information for every job and allows one organization to publish multiple opportunities. Form validation helps ensure that required values are supplied before data is saved. Once stored, the records become available to the relevant modules, including student job browsing, eligibility checking, recruiter candidate review and placement-office reporting.

STUDENT, COMPANY AND JOB ENTRY PAGE 21

The entry forms capture the core records needed to operate the placement workflow. Student details are associated with the registered user account and department, while company records describe the organization offering opportunities. Job entries are associated with a company and include the role description, application deadline and eligibility conditions. Keeping these records separate avoids duplicating company information for every job and allows one organization to publish multiple opportunities. Form validation helps ensure that required values are supplied before data is saved. Once stored, the records become available to the relevant modules, including student job browsing, eligibility checking, recruiter candidate review and placement-office reporting.


localhost:8080/CPMS/student/profile
CPMS
STUDENT
My Profile Settings
← Dashboard
NAVIGATION
Dashboard
My Profile
Academic Info
Job Openings
Applications
Resume Manager
Interviews
Messages
Notifications
Personal & Contact Information
Full Name
Arun Kumar
Name linked to your college registration record.
Register Number
CPMS2026001
Official college roll number.
College Email Address
student@cpms.test
Phone Number *
9000000000
Date of Birth
dd-mm-yyyy
Current Address
Residential / Communication address
Save Changes
Cancel
Academic Record
Academic details are officially verified by the Placement Cell.
Department
Computer Science and Engineering
Batch Year
2026
Verified CGPA
8.25
Active Backlogs
0
Need to update your CGPA or backlog count?
Submit Academic Update
Arun Kumar
CPMS2026001



controller/RegisterServlet.java

```
package com.cpms.controller;

@WebServlet("/register")
public class RegisterServlet extends HttpServlet {

   private AuthService authService;
   private DepartmentDAO departmentDAO;
   private CompanyDAO companyDAO;

   @Override
   public void init() {
      authService = new AuthService();
      departmentDAO = new DepartmentDAO();
      companyDAO = new CompanyDAO();
   }

   @Override
   protected void doGet(HttpServletRequest request, HttpServletResponse response)
         throws ServletException, IOException {

      List<Department> departments = departmentDAO.getAllDepartments();
      List<Company> companies = companyDAO.getActiveCompanies();

      request.setAttribute("departments", departments);
      request.setAttribute("companies", companies);

      request.getRequestDispatcher("/register.jsp").forward(request, response);
   }

   @Override
   protected void doPost(HttpServletRequest request, HttpServletResponse response)
         throws ServletException, IOException {
```

```
        String role = request.getParameter("role");
        String fullName = request.getParameter("fullName");
        String email = request.getParameter("email");
        String password = request.getParameter("password");
        String confirmPassword = request.getParameter("confirmPassword");

        String result = "";

        if ("STUDENT".equalsIgnoreCase(role)) {
            String registerNumber = request.getParameter("registerNumber");
            int departmentId = 0;
            int batchYear = 0;
            try {
                departmentId = Integer.parseInt(request.getParameter("departmentId"));
                batchYear = Integer.parseInt(request.getParameter("batchYear"));
            } catch (NumberFormatException ignored) {}

            result = authService.registerStudent(fullName, email, password, confirmPassword, registerNumber, departmentId, batchYear);

        } else if ("RECRUITER".equalsIgnoreCase(role)) {
            int companyId = 0;
            try {
                String compIdStr = request.getParameter("companyId");
                if (compIdStr != null && !compIdStr.isBlank()) {
                    companyId = Integer.parseInt(compIdStr);
                }
            } catch (NumberFormatException ignored) {}

            String newCompanyName = request.getParameter("newCompanyName");
            String designation = request.getParameter("designation");

            result = authService.registerRecruiter(fullName, email, password, confirmPassword, companyId, newCompanyName, designation);

        } else if ("OFFICER".equalsIgnoreCase(role)) {
            String employeeCode = request.getParameter("employeeCode");
            String designation = request.getParameter("designation");
            String inviteCode = request.getParameter("inviteCode");

            result = authService.registerOfficer(fullName, email, password, confirmPassword, employeeCode, designation, inviteCode);

        } else {
            result = "Invalid registration role specified.";
        }

        if ("SUCCESS".equals(result)) {
            response.sendRedirect(request.getContextPath() + "/login.jsp?registered=true");
        } else {
            request.setAttribute("errorMessage", result);
            request.setAttribute("selectedRole", role);
            request.setAttribute("fullName", fullName);
            request.setAttribute("email", email);
            doGet(request, response);
        }
    }
}
```

controller/OfficerJobServlet.java

```
        request.setAttribute("jobs", jobs);
        request.setAttribute("departments", departments);
        request.setAttribute("companies", companies);
        request.getRequestDispatcher("/officer/jobs.jsp").forward(request, response);
    }

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        HttpSession session = request.getSession(false);
        int userId = (Integer) session.getAttribute("userId");
        PlacementOfficer officer = officerDAO.findByUserId(userId);
        int officerId = officer != null ? officer.getOfficerId() : 1;

        int jobId = 0;
        try {
```

```
            String jobIdStr = request.getParameter("jobId");
            if (jobIdStr != null && !jobIdStr.isBlank()) {
                jobId = Integer.parseInt(jobIdStr);
            }
        } catch (NumberFormatException ignored) {}

        int companyId = Integer.parseInt(request.getParameter("companyId"));
        String jobTitle = request.getParameter("jobTitle");
        String jobDescription = request.getParameter("jobDescription");
        String jobType = request.getParameter("jobType");
        String location = request.getParameter("location");
        String minCgpaStr = request.getParameter("minCgpa");
        String maxBacklogsStr = request.getParameter("maxBacklogs");
        String packageLpaStr = request.getParameter("packageLpa");
        String deadlineStr = request.getParameter("applicationDeadline");
        String jobStatus = request.getParameter("jobStatus");

        String[] deptParam = request.getParameterValues("departmentIds");
        List<Integer> departmentIds = new ArrayList<>();
        if (deptParam != null) {
            for (String d : deptParam) {
                try { departmentIds.add(Integer.parseInt(d)); } catch (NumberFormatException ignored) {}
            }
        }

        Job job = new Job();
        job.setJobId(jobId);
        job.setCompanyId(companyId);
        job.setPostedBy(officerId);
        job.setJobTitle(jobTitle);
        job.setJobDescription(jobDescription);
        job.setJobType(jobType != null ? jobType : "FULL_TIME");
        job.setLocation(location);
        job.setMinCgpa(minCgpaStr != null && !minCgpaStr.isBlank() ? new BigDecimal(minCgpaStr) : BigDecimal.ZERO);
        job.setMaxBacklogs(maxBacklogsStr != null && !maxBacklogsStr.isBlank() ? Integer.parseInt(maxBacklogsStr) : 0);
        job.setPackageLpa(packageLpaStr != null && !packageLpaStr.isBlank() ? new BigDecimal(packageLpaStr) : null);

        if (deadlineStr != null && !deadlineStr.isBlank()) {
            try {
                // Parse HTML5 datetime-local or yyyy-MM-dd
                LocalDateTime ldt;
                if (deadlineStr.contains("T")) {
                    ldt = LocalDateTime.parse(deadlineStr);
                } else {
                    ldt = LocalDateTime.parse(deadlineStr + "T23:59:59");
                }
                job.setApplicationDeadline(Timestamp.valueOf(ldt));
            } catch (Exception ignored) {}
        }

        job.setJobStatus(jobStatus != null ? jobStatus : "OPEN");

        boolean saved = officerService.saveJob(job, departmentIds);
        if (saved) {
            response.sendRedirect(request.getContextPath() + "/officer/jobs?success=saved");
        } else {
            request.setAttribute("errorMessage", "Failed to save job posting.");
            doGet(request, response);
        }
    }
}
```

5.6 Placement officer dashboard

The placement officer dashboard provides an overview of the placement activities managed by the institution. It supports access to student records, company information, job postings, application progress and placement-related reports. The dashboard acts as the administrative entry point for coordinating opportunities across departments. By bringing these functions into one workspace, the system reduces the need to move between separate files or manually reconcile multiple lists. The officer remains responsible for maintaining accurate company and opportunity information.

controller/OfficerDashboardServlet.java

```
package com.cpms.controller;

@WebServlet("/officer/dashboard")
public class OfficerDashboardServlet extends HttpServlet {

    private OfficerService officerService;

    @Override
    public void init() {
        officerService = new OfficerService();
    }

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        HttpSession session = request.getSession(false);
        if (session == null || session.getAttribute("userId") == null) {
            response.sendRedirect(request.getContextPath() + "/login.jsp");
            return;
        }

        int userId = (Integer) session.getAttribute("userId");
        PlacementOfficer officer = officerService.getOfficerByUserId(userId);

        if (officer != null) {
            session.setAttribute("officerId", officer.getOfficerId());
            request.setAttribute("officer", officer);
        }

        Map<String, Object> metrics = officerService.getDashboardMetrics();
        List<AcademicUpdate> pendingUpdates = officerService.getPendingAcademicUpdates();
        List<Interview> upcomingInterviews = officerService.getUpcomingInterviews(5);
        List<Job> activeJobs = officerService.getAllJobs();
        List<Map<String, Object>> deptStats = officerService.getDepartmentPlacementStats();

        request.setAttribute("metrics", metrics);
        request.setAttribute("pendingUpdates", pendingUpdates);
        request.setAttribute("upcomingInterviews", upcomingInterviews);
        request.setAttribute("activeJobs", activeJobs);
        request.setAttribute("deptStats", deptStats);

        request.getRequestDispatcher("/officer/dashboard.jsp").forward(request, response);
    }
}
```

5.7 Student directory and record review

The student directory provides the placement team with a consolidated view of registered student information. It supports review of student identity, department and academic details needed for placement coordination. Centralized records help the officer identify missing or outdated information before opportunities are announced. The directory also provides a common reference when reviewing eligibility or application activity. Access to this administrative view is restricted to the appropriate role rather than being exposed to every logged-in user.

controller/OfficerStudentServlet.java

```
package com.cpms.controller;

@WebServlet("/officer/students")
public class OfficerStudentServlet extends HttpServlet {

    private OfficerService officerService;
    private DepartmentDAO departmentDAO;

    @Override
    public void init() {
        officerService = new OfficerService();
        departmentDAO = new DepartmentDAO();
    }

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        String query = request.getParameter("q");
        String deptStr = request.getParameter("departmentId");
        String batchStr = request.getParameter("batchYear");

        Integer departmentId = null;
        Integer batchYear = null;

        try {
            if (deptStr != null && !deptStr.isBlank()) departmentId = Integer.parseInt(deptStr);
            if (batchStr != null && !batchStr.isBlank()) batchYear = Integer.parseInt(batchStr);
        } catch (NumberFormatException ignored) {}

        List<Student> students = officerService.searchStudents(query, departmentId, batchYear);
        List<Department> departments = departmentDAO.getAllDepartments();

        request.setAttribute("students", students);
        request.setAttribute("departments", departments);
        request.setAttribute("query", query);
        request.setAttribute("selectedDept", departmentId);
        request.setAttribute("selectedBatch", batchYear);

        request.getRequestDispatcher("/officer/students.jsp").forward(request, response);
    }
}
```

5.8 Company and recruiter management

Company and recruiter information is maintained separately from student records so that organization details and recruiter access can be managed consistently. The placement office can maintain company information and associate opportunities with the relevant organization. Recruiter accounts provide access to candidate and application information connected with that organization's job postings. This separation supports clearer ownership of recruitment activity and helps prevent unrelated organizations from viewing one another's candidate pipelines.

dao/CompanyDAO.java

```
            e.printStackTrace();
        }
        return null;
    }

    public int createCompany(Company c) {
        String sql = """
            INSERT INTO companies (company_name, website, industry, description, contact_person, contact_email, contact_phone, is_active)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?)
        """;

        try (Connection con = DBConnection.getConnection();
             PreparedStatement ps = con.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {

            ps.setString(1, c.getCompanyName());
            ps.setString(2, c.getWebsite());
            ps.setString(3, c.getIndustry());
            ps.setString(4, c.getDescription());
            ps.setString(5, c.getContactPerson());
            ps.setString(6, c.getContactEmail());
            ps.setString(7, c.getContactPhone());
            ps.setBoolean(8, c.isActive());

            ps.executeUpdate();
            try (ResultSet rs = ps.getGeneratedKeys()) {
                if (rs.next()) {
                    return rs.getInt(1);
                }
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return -1;
    }

    public boolean updateCompany(Company c) {
        String sql = """
            UPDATE companies
            SET company_name = ?, website = ?, industry = ?, description = ?,
                contact_person = ?, contact_email = ?, contact_phone = ?, is_active = ?
            WHERE company_id = ?
        """;
```

controller/OfficerCompanyServlet.java

```
    @Override
    public void init() {
        officerService = new OfficerService();
    }

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        String toggleId = request.getParameter("toggle");
        if (toggleId != null) {
            try {
                int cId = Integer.parseInt(toggleId);
                boolean active = Boolean.parseBoolean(request.getParameter("active"));
                officerService.toggleCompanyStatus(cId, active);
                response.sendRedirect(request.getContextPath() + "/officer/companies?success=status");
                return;
            } catch (Exception ignored) {}
        }

        List<Company> companies = officerService.getAllCompanies();
        request.setAttribute("companies", companies);
        request.getRequestDispatcher("/officer/companies.jsp").forward(request, response);
    }

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
```

```
        int companyId = 0;
        try {
            String cIdStr = request.getParameter("companyId");
            if (cIdStr != null && !cIdStr.isBlank()) {
                companyId = Integer.parseInt(cIdStr);
            }
        } catch (NumberFormatException ignored) {}

        String companyName = request.getParameter("companyName");
        String website = request.getParameter("website");
        String industry = request.getParameter("industry");
        String description = request.getParameter("description");
        String contactPerson = request.getParameter("contactPerson");
        String contactEmail = request.getParameter("contactEmail");
        String contactPhone = request.getParameter("contactPhone");

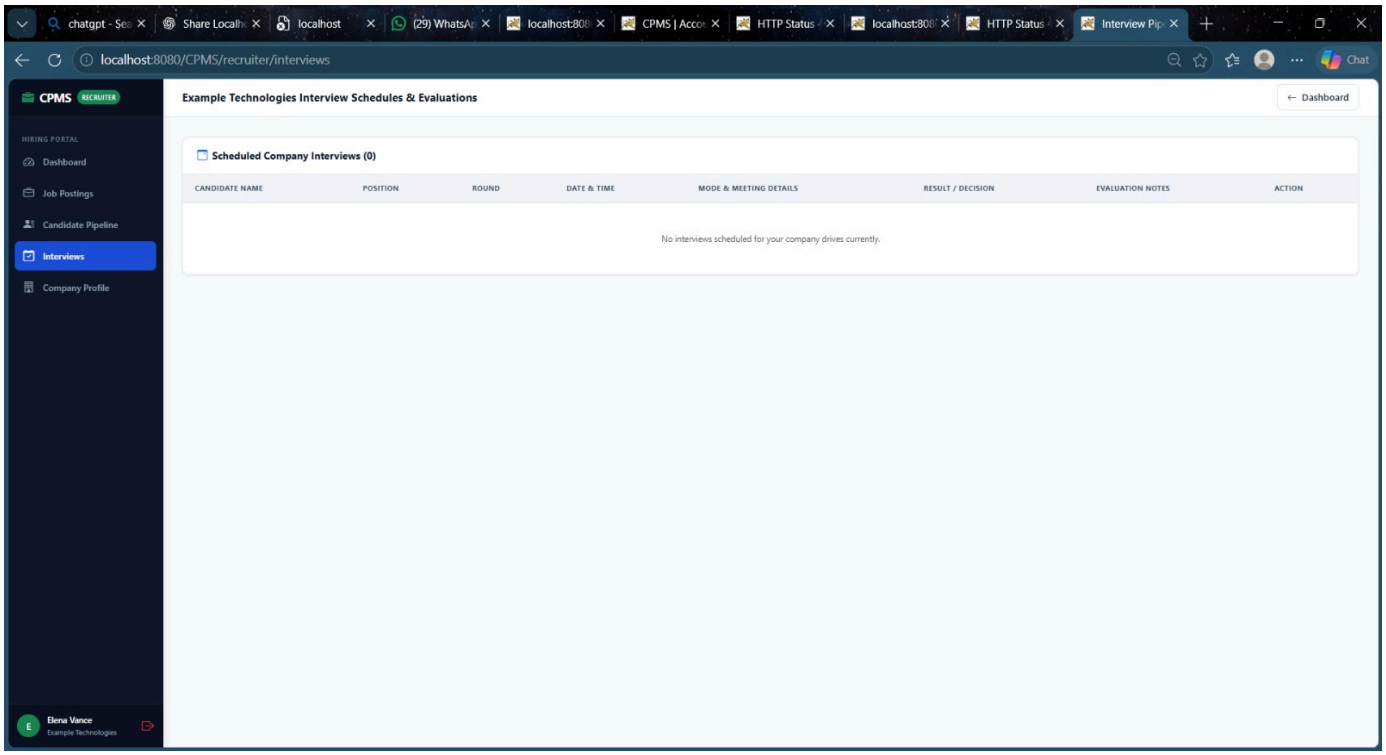
        if (companyName == null || companyName.isBlank()) {
            request.setAttribute("errorMessage", "Company name is required.");
            doGet(request, response);
            return;
        }

        Company c = new Company();
        c.setCompanyId(companyId);
        c.setCompanyName(companyName.trim());
        c.setWebsite(website != null ? website.trim() : null);
        c.setIndustry(industry != null ? industry.trim() : null);
        c.setDescription(description != null ? description.trim() : null);
        c.setContactPerson(contactPerson != null ? contactPerson.trim() : null);
        c.setContactEmail(contactEmail != null ? contactEmail.trim() : null);
        c.setContactPhone(contactPhone != null ? contactPhone.trim() : null);
        c.setActive(true);

        boolean saved = officerService.saveCompany(c);
        if (saved) {
            response.sendRedirect(request.getContextPath() + "/officer/companies?success=saved");
        } else {
            request.setAttribute("errorMessage", "Failed to save company details.");
            doGet(request, response);
        }
    }
}
```

5.9 Job posting and eligibility criteria

A job posting records the opportunity details together with its application deadline and eligibility conditions. The conditions can include a minimum CGPA, backlog restrictions and eligible departments. When a student views an opportunity, the system evaluates the stored academic information against the job requirements and presents the result as an eligibility outcome. This provides a consistent first-level check and helps students focus on opportunities for which they satisfy the configured criteria.



dao/JobDAO.java

```
package com.cpms.dao;

public class JobDAO {

  public int createJob(Job job, List<Integer> departmentIds) {
    String insertJobSql = """
      INSERT INTO jobs (company_id, posted_by, job_title, job_description, job_type, location,
                  min_cgpa, max_backlogs, package_lpa, application_deadline, job_status)
      VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
    """;

    Connection con = null;
    try {
      con = DBConnection.getConnection();
      con.setAutoCommit(false);

      int jobId = -1;
      try (PreparedStatement ps = con.prepareStatement(insertJobSql, Statement.RETURN_GENERATED_KEYS)) {
        ps.setInt(1, job.getCompanyId());
        ps.setInt(2, job.getPostedBy());
        ps.setString(3, job.getJobTitle());
        ps.setString(4, job.getJobDescription());
        ps.setString(5, job.getJobType());
        ps.setString(6, job.getLocation());
        ps.setBigDecimal(7, job.getMinCgpa());
        ps.setInt(8, job.getMaxBacklogs());
        ps.setBigDecimal(9, job.getPackageLpa());
        ps.setTimestamp(10, job.getApplicationDeadline());
        ps.setString(11, job.getJobStatus());

        ps.executeUpdate();
        try (ResultSet rs = ps.getGeneratedKeys()) {
          if (rs.next()) {
            jobId = rs.getInt(1);
            job.setJobId(jobId);
          }
        }
      }

      if (jobId > 0 && departmentIds != null && !departmentIds.isEmpty()) {
        String insertDeptSql = "INSERT INTO job_departments (job_id, department_id) VALUES (?, ?)";
        try (PreparedStatement psDept = con.prepareStatement(insertDeptSql)) {
          for (int deptId : departmentIds) {
            psDept.setInt(1, jobId);
            psDept.setInt(2, deptId);
            psDept.addBatch();
```

```
            }
            psDept.executeBatch();
        }
    }

    con.commit();
    return jobId;
```

service/EligibilityService.java (eligibility check)

```
public EligibilityService() {
    this.jobDAO = new JobDAO();
}

public EligibilityResult checkEligibility(Student student, Job job) {
    EligibilityResult result = new EligibilityResult();

    if (student == null) {
        result.addReason("Student record not found.");
        return result;
    }

    if (job == null) {
        result.addReason("Job posting not found.");
        return result;
    }

    // 1. Check job status
    if (!"OPEN".equalsIgnoreCase(job.getJobStatus())) {
        result.addReason("Job posting is closed.");
    }

    // 2. Check application deadline
    if (job.getApplicationDeadline() != null &&
        job.getApplicationDeadline().before(new java.util.Date())) {
        result.addReason("Application deadline has passed.");
    }

    // 3. Check CGPA (Strictly using verified student CGPA)
    BigDecimal studentCgpa = student.getCgpa() != null ? student.getCgpa() : BigDecimal.ZERO;
    BigDecimal minCgpa = job.getMinCgpa() != null ? job.getMinCgpa() : BigDecimal.ZERO;

    if (studentCgpa.compareTo(minCgpa) < 0) {
        result.addReason("CGPA requirement not met (Required: " + minCgpa + ", Yours: " + studentCgpa + ").");
    }

    // 4. Check active backlogs (Strictly using verified student backlogs)
    int studentBacklogs = student.getActiveBacklogs();
    int maxBacklogs = job.getMaxBacklogs();

    if (studentBacklogs > maxBacklogs) {
        result.addReason("Backlog limit exceeded (Maximum allowed: " + maxBacklogs + ", Yours: " + studentBacklogs + ").");
    }

    // 5. Check eligible departments
    List<Integer> allowedDeptIds = job.getDepartmentIds();
    if (allowedDeptIds == null || allowedDeptIds.isEmpty()) {
        allowedDeptIds = jobDAO.getDepartmentIdsForJob(job.getJobId());
    }

    if (allowedDeptIds != null && !allowedDeptIds.isEmpty()) {
        if (!allowedDeptIds.contains(student.getDepartmentId())) {
            result.addReason("Your department (" + (student.getDepartmentCode() != null ? student.getDepartmentCode() : "Dept ID " +
student.getDepartmentId()) +
                    ") is not eligible for this role.");
        }
    }

    return result;
}
}
```

5.10 Application tracking

The application module records a student's application to a job and maintains its current status as the recruitment process advances. Students can review the progress of their submissions, while authorized placement staff and recruiters can manage the stages relevant to their roles. Maintaining the application as a database record creates a traceable link between the student, the job and subsequent interview or offer information. It also supports reporting on application volume and recruitment progress.

dao/ApplicationDAO.java

```
package com.cpms.dao;

public class ApplicationDAO {

    public boolean apply(int studentId, int jobId, Integer resumeId) {
        String sql = """
            INSERT INTO applications (student_id, job_id, resume_id, application_status)
            VALUES (?, ?, ?, 'APPLIED')
        """;

        try (Connection con = DBConnection.getConnection();
             PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setInt(1, studentId);
            ps.setInt(2, jobId);
            if (resumeId != null && resumeId > 0) {
                ps.setInt(3, resumeId);
            } else {
                ps.setNull(3, Types.INTEGER);
            }

            return ps.executeUpdate() > 0;
        } catch (SQLException e) {
            e.printStackTrace();
            return false;
        }
    }

    public boolean withdraw(int applicationId, int studentId) {
        String sql = "UPDATE applications SET application_status = 'WITHDRAWN' WHERE application_id = ? AND student_id = ? AND
application_status IN ('APPLIED', 'UNDER_REVIEW')";
        try (Connection con = DBConnection.getConnection();
             PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setInt(1, applicationId);
            ps.setInt(2, studentId);
            return ps.executeUpdate() > 0;
        } catch (SQLException e) {
            e.printStackTrace();
            return false;
        }
    }

    public Application findByStudentAndJob(int studentId, int jobId) {
        String sql = """
            SELECT a.*, u.full_name as student_name, s.register_number, u.email as student_email,
                   d.department_name, s.batch_year, s.cgpa, s.active_backlogs,
                   j.job_title, c.company_name, j.package_lpa, j.location,
                   r.file_name as resume_file_name, r.file_path as resume_file_path
            FROM applications a
            JOIN students s ON a.student_id = s.student_id
            JOIN users u ON s.user_id = u.user_id
            JOIN departments d ON s.department_id = d.department_id
            JOIN jobs j ON a.job_id = j.job_id
            JOIN companies c ON j.company_id = c.company_id
            LEFT JOIN resumes r ON a.resume_id = r.resume_id
            WHERE a.student_id = ? AND a.job_id = ?
        """;

        try (Connection con = DBConnection.getConnection();
             PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setInt(1, studentId);
            ps.setInt(2, jobId);
```

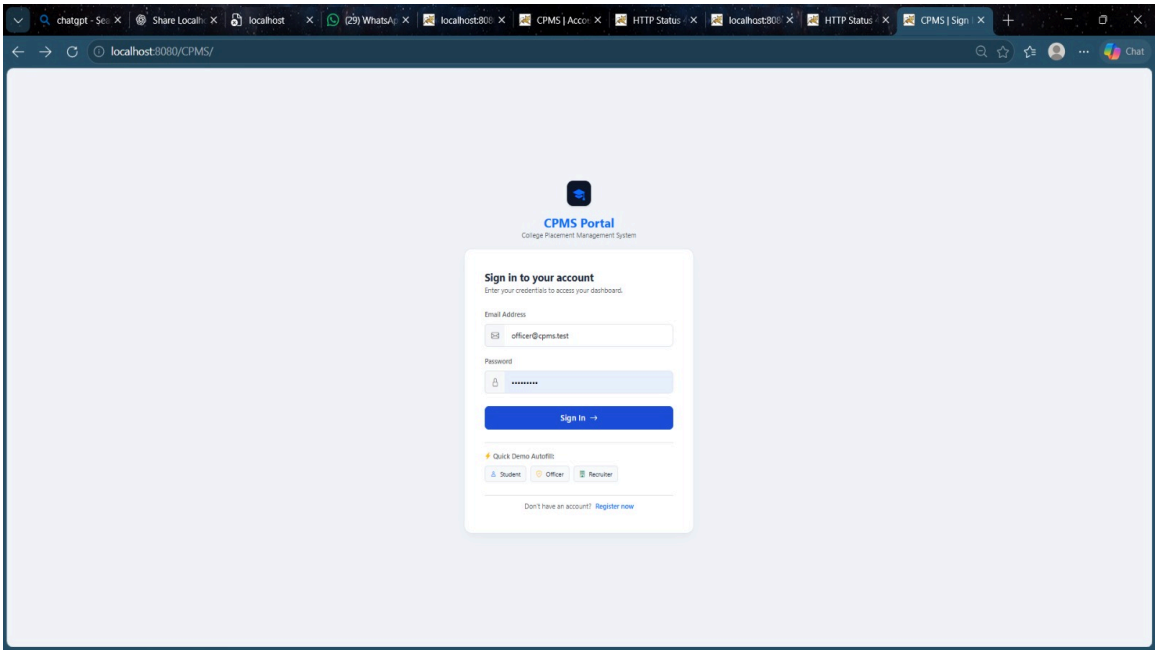
5.4 DASHBOARDS AND RECORDS VIEW PAGE 27

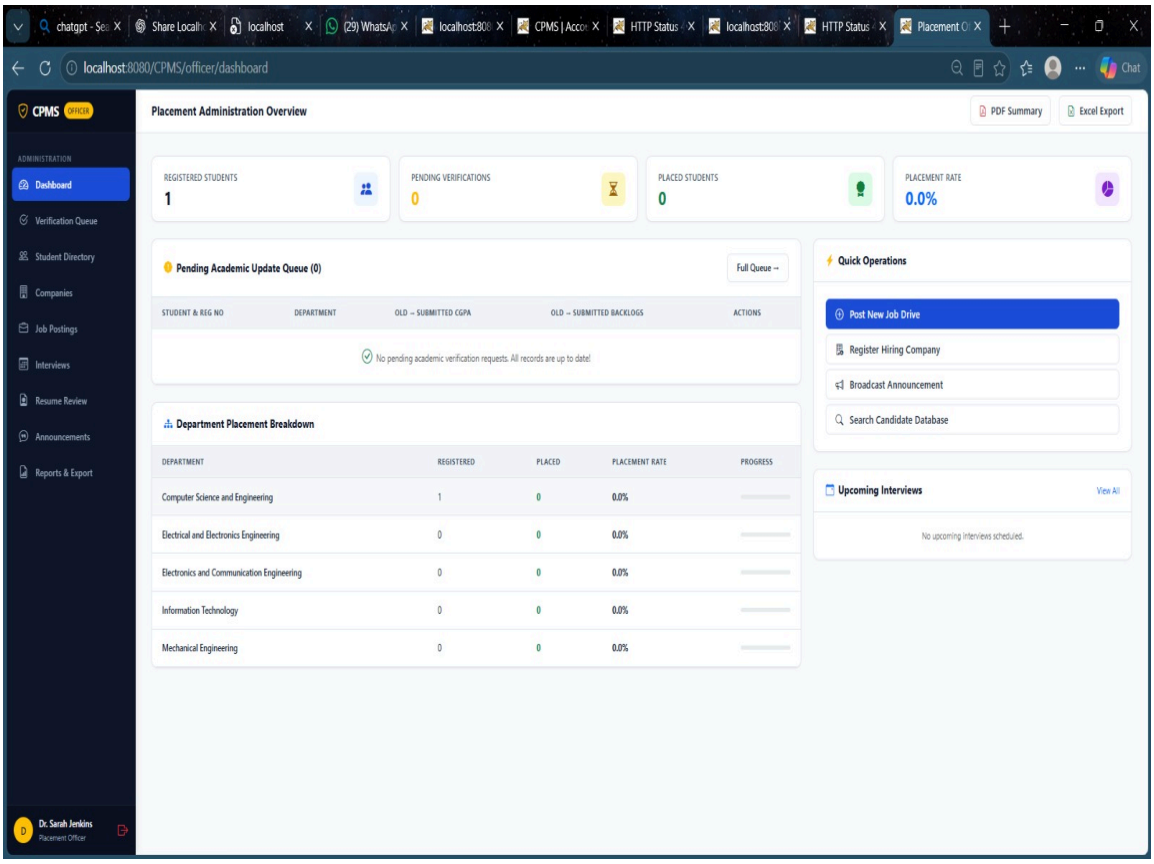
The dashboard and records views provide role-specific summaries of information already stored in the database. The student view focuses on personal placement activity, including profile information, available opportunities and application progress. The placement officer view provides broader access to student, company and job records so that placement activity can be coordinated across departments. Recruiter views focus on candidates who have applied to the recruiter's organization's opportunities. Tables and summary panels make it easier to review records without opening each item individually. The views are designed around the tasks of each role, while the underlying data remains

connected through database relationships.

DASHBOARDS AND RECORDS VIEW PAGE 27

The dashboard and records views provide role-specific summaries of information already stored in the database. The student view focuses on personal placement activity, including profile information, available opportunities and application progress. The placement officer view provides broader access to student, company and job records so that placement activity can be coordinated across departments. Recruiter views focus on candidates who have applied to the recruiter's organization's opportunities. Tables and summary panels make it easier to review records without opening each item individually. The views are designed around the tasks of each role, while the underlying data remains connected through database relationships.




localhost:8080/CPMS/officer/dashboard
CPMS
OFFICER
ADMINISTRATION
Dashboard
Verification Queue
Student Directory
Companies
Job Postings
Interviews
Resume Review
Announcements
Reports & Export
Dr. Sarah Jenkins
Placement Officer
Placement Administration Overview
PDF Summary
Excel Export
REGISTERED STUDENTS
1
PENDING VERIFICATIONS
0
PLACED STUDENTS
0
PLACEMENT RATE
0.0%
Pending Academic Update Queue (0)
Full Queue →
STUDENT & REG NO
DEPARTMENT
OLD → SUBMITTED CGPA
OLD → SUBMITTED BACKLOGS
ACTIONS
No pending academic verification requests. All records are up to date!
Quick Operations
Post New Job Drive
Register Hiring Company
Broadcast Announcement
Search Candidate Database
Department Placement Breakdown
DEPARTMENT
REGISTERED
PLACED
PLACEMENT RATE
PROGRESS
Computer Science and Engineering 1 0 0.0%
Electrical and Electronics Engineering 0 0 0.0%
Electronics and Communication Engineering 0 0 0.0%
Information Technology 0 0 0.0%
Mechanical Engineering 0 0 0.0%
Upcoming Interviews
View All
No upcoming interviews scheduled.

controller/StudentDashboardServlet.java

```
package com.cpms.controller;

@WebServlet("/student/dashboard")
public class StudentDashboardServlet extends HttpServlet {

    private StudentService studentService;
    private NotificationService notificationService;

    @Override
    public void init() {
        studentService = new StudentService();
        notificationService = new NotificationService();
    }

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        HttpSession session = request.getSession(false);
        if (session == null || session.getAttribute("userId") == null) {
            response.sendRedirect(request.getContextPath() + "/login.jsp");
            return;
        }

        int userId = (Integer) session.getAttribute("userId");
        Student student = studentService.getStudentProfile(userId);

        if (student == null) {
            request.setAttribute("errorMessage", "Student record not linked. Please contact the Placement Cell.");
            request.getRequestDispatcher("/student/dashboard.jsp").forward(request, response);
            return;
        }

        // Cache studentId in session if not present
        session.setAttribute("studentId", student.getStudentId());

        List<Job> availableJobs = studentService.getAvailableJobs(student);
        List<Application> applications = studentService.getStudentApplications(student.getStudentId());
        List<Interview> interviews = studentService.getStudentInterviews(student.getStudentId());
        List<Notification> notifications = notificationService.getUserNotifications(userId, false);
        Resume activeResume = studentService.getActiveResume(student.getStudentId());
        List<AcademicUpdate> academicHistory = studentService.getAcademicHistory(student.getStudentId());

        // Compute application statistics
        int totalApps = applications.size();
        int shortlistedApps = 0;
        int selectedApps = 0;
        int rejectedApps = 0;

        for (Application app : applications) {
            String status = app.getApplicationStatus();
            if ("SHORTLISTED".equalsIgnoreCase(status)) shortlistedApps++;
            else if ("SELECTED".equalsIgnoreCase(status)) selectedApps++;
            else if ("REJECTED".equalsIgnoreCase(status)) rejectedApps++;
        }

        // Compute profile completion percentage
        int profilePoints = 0;
        if (student.getFullName() != null && !student.getFullName().isBlank()) profilePoints += 20;
        if (student.getPhone() != null && !student.getPhone().isBlank()) profilePoints += 20;
        if (student.getDateOfBirth() != null) profilePoints += 20;
        if (student.getAddress() != null && !student.getAddress().isBlank()) profilePoints += 20;
        if (activeResume != null) profilePoints += 20;

        request.setAttribute("student", student);
        request.setAttribute("availableJobs", availableJobs);
        request.setAttribute("applications", applications);
        request.setAttribute("interviews", interviews);
        request.setAttribute("notifications", notifications);
        request.setAttribute("activeResume", activeResume);
        request.setAttribute("academicHistory", academicHistory);
        request.setAttribute("totalApps", totalApps);
        request.setAttribute("shortlistedApps", shortlistedApps);
        request.setAttribute("selectedApps", selectedApps);
        request.setAttribute("rejectedApps", rejectedApps);
```

```
        request.setAttribute("profileCompletion", profilePoints);

        request.getRequestDispatcher("/student/dashboard.jsp").forward(request, response);
    }
}
```

controller/RecruiterDashboardServlet.java

```
package com.cpms.controller;

@WebServlet("/recruiter/dashboard")
public class RecruiterDashboardServlet extends HttpServlet {

    private RecruiterService recruiterService;

    @Override
    public void init() {
        recruiterService = new RecruiterService();
    }

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        HttpSession session = request.getSession(false);
        if (session == null || session.getAttribute("userId") == null) {
            response.sendRedirect(request.getContextPath() + "/login.jsp");
            return;
        }

        int userId = (Integer) session.getAttribute("userId");
        Recruiter recruiter = recruiterService.getRecruiterByUserId(userId);

        if (recruiter == null) {
            request.setAttribute("errorMessage", "Recruiter profile not linked to a company.");
            request.getRequestDispatcher("/recruiter/dashboard.jsp").forward(request, response);
            return;
        }

        session.setAttribute("companyId", recruiter.getCompanyId());
        session.setAttribute("companyName", recruiter.getCompanyName());

        Map<String, Object> metrics = recruiterService.getRecruiterDashboardMetrics(recruiter.getCompanyId());
        List<Job> jobs = recruiterService.getCompanyJobs(recruiter.getCompanyId());
        List<Application> applicants = recruiterService.getApplicantsForCompany(recruiter.getCompanyId());
        List<Interview> interviews = recruiterService.getCompanyInterviews(recruiter.getCompanyId());

        request.setAttribute("recruiter", recruiter);
        request.setAttribute("metrics", metrics);
        request.setAttribute("jobs", jobs);
        request.setAttribute("applicants", applicants);
        request.setAttribute("interviews", interviews);

        request.getRequestDispatcher("/recruiter/dashboard.jsp").forward(request, response);
    }
}
```

5.11 Recruiter workspace

The recruiter workspace focuses on the opportunities and candidates associated with the recruiter's organization. It provides access to candidate information and the application pipeline for the recruiter's job postings. The recruiter can review submissions and follow the recruitment stages supported by the application. Keeping this workspace separate from the placement officer dashboard clarifies the responsibilities of the organization and the institution while allowing both parties to work with the same underlying placement records.

controller/RecruiterJobServlet.java

```
package com.cpms.controller;

@WebServlet("/recruiter/jobs")
public class RecruiterJobServlet extends HttpServlet {

    private RecruiterService recruiterService;
    private DepartmentDAO departmentDAO;
    private JobDAO jobDAO;

    @Override
    public void init() {
        recruiterService = new RecruiterService();
        departmentDAO = new DepartmentDAO();
        jobDAO = new JobDAO();
    }

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        HttpSession session = request.getSession(false);
        int userId = (Integer) session.getAttribute("userId");
        Recruiter recruiter = recruiterService.getRecruiterByUserId(userId);

        if (recruiter == null) {
            response.sendRedirect(request.getContextPath() + "/recruiter/dashboard");
            return;
        }

        List<Job> jobs = recruiterService.getCompanyJobs(recruiter.getCompanyId());
        List<Department> departments = departmentDAO.getAllDepartments();

        request.setAttribute("recruiter", recruiter);
        request.setAttribute("jobs", jobs);
        request.setAttribute("departments", departments);

        request.getRequestDispatcher("/recruiter/jobs.jsp").forward(request, response);
    }

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        HttpSession session = request.getSession(false);
        int userId = (Integer) session.getAttribute("userId");
        Recruiter recruiter = recruiterService.getRecruiterByUserId(userId);

        if (recruiter == null) {
            response.sendRedirect(request.getContextPath() + "/recruiter/dashboard");
            return;
        }

        String jobTitle = request.getParameter("jobTitle");
        String jobDescription = request.getParameter("jobDescription");
        String jobType = request.getParameter("jobType");
        String location = request.getParameter("location");
        String minCgpaStr = request.getParameter("minCgpa");
        String maxBacklogsStr = request.getParameter("maxBacklogs");
        String packageLpaStr = request.getParameter("packageLpa");
        String deadlineStr = request.getParameter("applicationDeadline");

        String[] deptParam = request.getParameterValues("departmentIds");
        List<Integer> departmentIds = new ArrayList<>();
        if (deptParam != null) {
            for (String d : deptParam) {
                try { departmentIds.add(Integer.parseInt(d)); } catch (NumberFormatException ignored) {}
            }
        }

        Job job = new Job();
        job.setCompanyId(recruiter.getCompanyId());
        job.setPostedBy(1); // Standard default placement cell officer ID
        job.setJobTitle(jobTitle);
        job.setJobDescription(jobDescription);
```

```
        job.setJobType(jobType != null ? jobType : "FULL_TIME");
        job.setLocation(location);
        job.setMinCgpa(minCgpaStr != null && !minCgpaStr.isBlank() ? new BigDecimal(minCgpaStr) : BigDecimal.ZERO);
        job.setMaxBacklogs(maxBacklogsStr != null && !maxBacklogsStr.isBlank() ? Integer.parseInt(maxBacklogsStr) : 0);
        job.setPackageLpa(packageLpaStr != null && !packageLpaStr.isBlank() ? new BigDecimal(packageLpaStr) : null);

        if (deadlineStr != null && !deadlineStr.isBlank()) {
            try {
                LocalDateTime ldt;
                if (deadlineStr.contains("T")) {
                    ldt = LocalDateTime.parse(deadlineStr);
                } else {
                    ldt = LocalDateTime.parse(deadlineStr + "T23:59:59");
                }
                job.setApplicationDeadline(Timestamp.valueOf(ldt));
            } catch (Exception ignored) {}
        }
        job.setJobStatus("OPEN");

        int jobId = jobDAO.createJob(job, departmentIds);
        if (jobId > 0) {
            response.sendRedirect(request.getContextPath() + "/recruiter/jobs?success=created");
        } else {
            request.setAttribute("errorMessage", "Failed to post job listing.");
            doGet(request, response);
        }
    }
}
```

controller/RecruiterCandidateServlet.java

```
package com.cpms.controller;

@WebServlet("/recruiter/candidates")
public class RecruiterCandidateServlet extends HttpServlet {

    private RecruiterService recruiterService;

    @Override
    public void init() {
        recruiterService = new RecruiterService();
    }

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        HttpSession session = request.getSession(false);
        int userId = (Integer) session.getAttribute("userId");
        Recruiter recruiter = recruiterService.getRecruiterByUserId(userId);

        if (recruiter == null) {
            response.sendRedirect(request.getContextPath() + "/recruiter/dashboard");
            return;
        }

        String viewStudentIdStr = request.getParameter("viewStudentId");
        if (viewStudentIdStr != null && !viewStudentIdStr.isBlank()) {
            int studentId = Integer.parseInt(viewStudentIdStr);
            Student candidate = recruiterService.getCandidateProfileForRecruiter(studentId, recruiter.getCompanyId());
            if (candidate == null) {
                response.sendError(HttpServletResponse.SC_FORBIDDEN, "Access denied: Candidate has not applied to your company.");
                return;
            }
            request.setAttribute("candidate", candidate);
            request.getRequestDispatcher("/recruiter/candidate-detail.jsp").forward(request, response);
            return;
        }

        List<Application> applicants = recruiterService.getApplicantsForCompany(recruiter.getCompanyId());
        List<Job> companyJobs = recruiterService.getCompanyJobs(recruiter.getCompanyId());

        request.setAttribute("applicants", applicants);
        request.setAttribute("jobs", companyJobs);
        request.setAttribute("recruiter", recruiter);
```

```
        request.getRequestDispatcher("/recruiter/candidates.jsp").forward(request, response);
    }

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        HttpSession session = request.getSession(false);
        int userId = (Integer) session.getAttribute("userId");
        Recruiter recruiter = recruiterService.getRecruiterByUserId(userId);

        if (recruiter == null) {
            response.sendRedirect(request.getContextPath() + "/recruiter/dashboard");
            return;
        }

        int applicationId = Integer.parseInt(request.getParameter("applicationId"));
        String status = request.getParameter("status"); // SHORTLISTED, REJECTED, SELECTED, UNDER_REVIEW

        boolean updated = recruiterService.updateCandidateStatus(applicationId, recruiter.getCompanyId(), status);

        if (updated) {
            response.sendRedirect(request.getContextPath() + "/recruiter/candidates?success=status");
        } else {
            request.setAttribute("errorMessage", "Failed to update candidate status or unauthorized access.");
            doGet(request, response);
        }
    }
}
```

5.12 Candidate pipeline and interview coordination

The candidate pipeline gives recruiters a structured view of applicants as they move through the selection process. Interview details can be associated with the relevant application so that the student, recruiter and placement team can refer to the same scheduled event. The interview record stores scheduling information such as date, time and mode, supporting coordination and follow-up. A centralized record reduces ambiguity that can occur when interview details are communicated through disconnected messages.

controller/RecruiterInterviewServlet.java

```
package com.cpms.controller;

@WebServlet("/recruiter/interviews")
public class RecruiterInterviewServlet extends HttpServlet {

    private RecruiterService recruiterService;
    private InterviewDAO interviewDAO;

    @Override
    public void init() {
        recruiterService = new RecruiterService();
        interviewDAO = new InterviewDAO();
    }

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        HttpSession session = request.getSession(false);
        int userId = (Integer) session.getAttribute("userId");
        Recruiter recruiter = recruiterService.getRecruiterByUserId(userId);

        if (recruiter == null) {
            response.sendRedirect(request.getContextPath() + "/recruiter/dashboard");
            return;
        }

        List<Interview> interviews = recruiterService.getCompanyInterviews(recruiter.getCompanyId());

        request.setAttribute("interviews", interviews);
        request.setAttribute("recruiter", recruiter);
        request.getRequestDispatcher("/recruiter/interviews.jsp").forward(request, response);
    }

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        HttpSession session = request.getSession(false);
        int userId = (Integer) session.getAttribute("userId");
        Recruiter recruiter = recruiterService.getRecruiterByUserId(userId);

        if (recruiter == null) {
            response.sendRedirect(request.getContextPath() + "/recruiter/dashboard");
            return;
        }

        int interviewId = Integer.parseInt(request.getParameter("interviewId"));
        String result = request.getParameter("result");
        String notes = request.getParameter("notes");

        // Verify interview belongs to this recruiter's company
        Interview iv = interviewDAO.findById(interviewId);
        if (iv == null || iv.getCompanyId() != recruiter.getCompanyId()) {
            response.sendError(HttpServletResponse.SC_FORBIDDEN, "Access denied.");
            return;
        }

        boolean updated = interviewDAO.updateInterviewResult(interviewId, result, notes);
        if (updated) {
            response.sendRedirect(request.getContextPath() + "/recruiter/interviews?success=result");
        } else {
            request.setAttribute("errorMessage", "Failed to update interview evaluation.");
            doGet(request, response);
        }
    }
}
```

dao/InterviewDAO.java

```
package com.cpms.dao;
```

```
public class InterviewDAO {

    public boolean scheduleInterview(Interview interview) {
        String sql = """
            INSERT INTO interviews (application_id, round_number, interview_datetime, mode, meeting_or_venue, scheduled_by, result, notes)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?)
        """;

        try (Connection con = DBConnection.getConnection();
             PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setInt(1, interview.getApplicationId());
            ps.setInt(2, interview.getRoundNumber());
            ps.setTimestamp(3, interview.getInterviewDatetime());
            ps.setString(4, interview.getMode());
            ps.setString(5, interview.getMeetingOrVenue());
            ps.setInt(6, interview.getScheduledBy());
            ps.setString(7, interview.getResult());
            ps.setString(8, interview.getNotes());

            return ps.executeUpdate() > 0;
        } catch (SQLException e) {
            e.printStackTrace();
            return false;
        }
    }

    public boolean updateInterviewResult(int interviewId, String result, String notes) {
        String sql = "UPDATE interviews SET result = ?, notes = ? WHERE interview_id = ?";
        try (Connection con = DBConnection.getConnection();
             PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setString(1, result);
            ps.setString(2, notes);
            ps.setInt(3, interviewId);

            return ps.executeUpdate() > 0;
        } catch (SQLException e) {
            e.printStackTrace();
            return false;
        }
```

5.13 Notifications and communication

The notification center provides in-application updates for events such as new job announcements, application status changes and interview reminders. Notifications are linked to the intended user so that information can be displayed in the correct role workspace. The messaging and notification features complement the formal application records: messages support communication, while notifications draw attention to important changes. This helps users stay informed without relying entirely on separate manual communication channels.

service/NotificationService.java

```
package com.cpms.service;

public class NotificationService {

    private final NotificationDAO notificationDAO;

    public NotificationService() {
        this.notificationDAO = new NotificationDAO();
    }

    public boolean notifyUser(int userId, String title, String message, String type) {
        if (userId <= 0 || title == null || message == null) return false;
        return notificationDAO.createNotification(userId, title, message, type);
    }

    public void notifyAllOfficers(String title, String message, String type) {
        String sql = "SELECT user_id FROM users WHERE role = 'OFFICER' AND is_active = TRUE";
        try (Connection con = DBConnection.getConnection();
             PreparedStatement ps = con.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {

            while (rs.next()) {
                notificationDAO.createNotification(rs.getInt("user_id"), title, message, type);
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }

    public void notifyAllStudents(String title, String message, String type) {
        String sql = "SELECT user_id FROM users WHERE role = 'STUDENT' AND is_active = TRUE";
        try (Connection con = DBConnection.getConnection();
             PreparedStatement ps = con.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {

            while (rs.next()) {
                notificationDAO.createNotification(rs.getInt("user_id"), title, message, type);
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }

    public List<Notification> getUserNotifications(int userId, boolean unreadOnly) {
        return notificationDAO.getNotificationsForUser(userId, unreadOnly);
    }

    public boolean markRead(int notificationId, int userId) {
        return notificationDAO.markAsRead(notificationId, userId);
    }

    public boolean markAllRead(int userId) {
        return notificationDAO.markAllAsRead(userId);
    }

    public int countUnread(int userId) {
        return notificationDAO.countUnread(userId);
    }
}
```

5.5 UPDATE STUDENT, COMPANY AND JOB RECORDS PAGE:

Update operations allow authorized users to correct or maintain information after an initial record has been created. Student academic information may need revision when verified results change; company information may need correction when contact or organization details are updated; and job details may change before the application deadline. The application retrieves the selected record, presents its current values and submits the revised information for persistence. Updating an existing record preserves its identity and relationships with associated applications or other records. Restricting administrative update functions to the placement officer helps maintain data quality and accountability.

UPDATE STUDENT, COMPANY AND JOB RECORDS PAGE 31

Update operations allow authorized users to correct or maintain information after an initial record has been created. Student academic information may need revision when verified results change; company information may need correction when contact or organization details are updated; and job details may change before the application deadline. The application retrieves the selected record, presents its current values and submits the revised information for persistence. Updating an existing record preserves its identity and relationships with associated applications or other records. Restricting administrative update functions to the placement officer helps maintain data quality and accountability.

controller/StudentAcademicServlet.java

```
package com.cpms.controller;

@WebServlet("/student/academic")
public class StudentAcademicServlet extends HttpServlet {

    private StudentService studentService;

    @Override
    public void init() {
        studentService = new StudentService();
    }

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        HttpSession session = request.getSession(false);
        if (session == null || session.getAttribute("userId") == null) {
            response.sendRedirect(request.getContextPath() + "/login.jsp");
            return;
        }

        int userId = (Integer) session.getAttribute("userId");
        Student student = studentService.getStudentProfile(userId);

        if (student == null) {
            response.sendRedirect(request.getContextPath() + "/student/dashboard");
            return;
        }

        List<AcademicUpdate> history = studentService.getAcademicHistory(student.getStudentId());

        request.setAttribute("student", student);
        request.setAttribute("history", history);
        request.getRequestDispatcher("/student/academic.jsp").forward(request, response);
    }

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        HttpSession session = request.getSession(false);
        if (session == null || session.getAttribute("userId") == null) {
            response.sendRedirect(request.getContextPath() + "/login.jsp");
            return;
        }

        int userId = (Integer) session.getAttribute("userId");
        Student student = studentService.getStudentProfile(userId);
        if (student == null) {
            response.sendRedirect(request.getContextPath() + "/student/dashboard");
            return;
        }

        String cgpaStr = request.getParameter("submittedCgpa");
        String backlogsStr = request.getParameter("submittedBacklogs");
        String remarks = request.getParameter("studentRemarks");

        BigDecimal submittedCgpa;
        int submittedBacklogs;
```

```
        try {
            submittedCgpa = new BigDecimal(cgpaStr);
            submittedBacklogs = Integer.parseInt(backlogsStr);
        } catch (Exception e) {
            request.setAttribute("errorMessage", "Invalid CGPA or backlog format.");
            doGet(request, response);
            return;
        }

        String result = studentService.submitAcademicUpdate(student.getStudentId(), submittedCgpa, submittedBacklogs, remarks);

        if ("SUCCESS".equals(result)) {
            response.sendRedirect(request.getContextPath() + "/student/academic?success=submitted");
        } else {
            request.setAttribute("errorMessage", result);
            doGet(request, response);
        }
    }
}
```

controller/OfficerVerificationServlet.java

```
package com.cpms.controller;

@WebServlet("/officer/verifications")
public class OfficerVerificationServlet extends HttpServlet {

    private OfficerService officerService;
    private PlacementOfficerDAO officerDAO;

    @Override
    public void init() {
        officerService = new OfficerService();
        officerDAO = new PlacementOfficerDAO();
    }

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        List<AcademicUpdate> pendingUpdates = officerService.getPendingAcademicUpdates();
        request.setAttribute("pendingUpdates", pendingUpdates);
        request.getRequestDispatcher("/officer/verifications.jsp").forward(request, response);
    }

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        HttpSession session = request.getSession(false);
        int userId = (Integer) session.getAttribute("userId");
        PlacementOfficer officer = officerDAO.findByUserId(userId);

        int officerId = officer != null ? officer.getOfficerId() : 1;

        int updateId = 0;
        try {
            updateId = Integer.parseInt(request.getParameter("updateId"));
        } catch (NumberFormatException ignored) {}

        String action = request.getParameter("action"); // APPROVE or REJECT
        String remarks = request.getParameter("officerRemarks");

        String status = "APPROVE".equalsIgnoreCase(action) ? "APPROVED" : "REJECTED";

        boolean reviewed = officerService.reviewAcademicUpdate(updateId, officerId, status, remarks);

        if (reviewed) {
            response.sendRedirect(request.getContextPath() + "/officer/verifications?success=" + status.toLowerCase());
        } else {
            request.setAttribute("errorMessage", "Failed to process academic verification.");
            doGet(request, response);
        }
```

```
  }
}
```

5.14 Reports and placement analytics

The reporting features help the placement office review activity recorded by the system. Reports can summarize job postings, student applications and placement outcomes, providing a consolidated view of the recruitment process. The report history feature supports traceability of generated reports. Export options make the information easier to share for academic review and institutional documentation. The accuracy of a report depends on the completeness of the underlying student, job and application records.

controller/OfficerReportServlet.java

```
package com.cpms.controller;

@WebServlet("/officer/reports")
public class OfficerReportServlet extends HttpServlet {

    private OfficerService officerService;
    private ReportService reportService;

    @Override
    public void init() {
        officerService = new OfficerService();
        reportService = new ReportService();
    }

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        HttpSession session = request.getSession(false);
        int userId = (Integer) session.getAttribute("userId");

        String format = request.getParameter("download");

        if ("pdf".equalsIgnoreCase(format)) {
            byte[] pdfBytes = reportService.generatePlacementSummaryPdf(userId);
            response.setContentType("application/pdf");
            response.setHeader("Content-Disposition", "attachment; filename=\"CPMS_Placement_Report_" + LocalDate.now() + ".pdf\"");
            response.setContentLength(pdfBytes.length);

            try (OutputStream out = response.getOutputStream()) {
                out.write(pdfBytes);
                out.flush();
            }
            return;

        } else if ("excel".equalsIgnoreCase(format)) {
            byte[] excelBytes = reportService.generatePlacementSummaryExcel(userId);
            response.setContentType("application/vnd.openxmlformats-officedocument.spreadsheetml.sheet");
            response.setHeader("Content-Disposition", "attachment; filename=\"CPMS_Placement_Report_" + LocalDate.now() + ".xlsx\"");
            response.setContentLength(excelBytes.length);

            try (OutputStream out = response.getOutputStream()) {
                out.write(excelBytes);
                out.flush();
            }
            return;
        }

        Map<String, Object> metrics = officerService.getDashboardMetrics();
        List<Map<String, Object>> deptStats = officerService.getDepartmentPlacementStats();
        List<Map<String, Object>> companyStats = officerService.getCompanyPlacements();

        request.setAttribute("metrics", metrics);
        request.setAttribute("deptStats", deptStats);
        request.setAttribute("companyStats", companyStats);
        request.getRequestDispatcher("/officer/reports.jsp").forward(request, response);
    }
}
```

service/ReportService.java

```
package com.cpms.service;

public class ReportService {

    private final ReportDAO reportDAO;

    public ReportService() {
        this.reportDAO = new ReportDAO();
    }

    public byte[] generatePlacementSummaryPdf(int officerUserId) throws IOException {
```

```
Map<String, Object> overall = reportDAO.getOverallPlacementStats();
List<Map<String, Object>> deptStats = reportDAO.getDepartmentWisePlacementStats();
List<Map<String, Object>> companyStats = reportDAO.getCompanyWisePlacements();

try (PDDocument document = new PDDocument()) {
   PDPage page = new PDPage(PDRectangle.A4);
   document.addPage(page);

   try (PDPageContentStream cs = new PDPageContentStream(document, page)) {
      PDType1Font boldFont = new PDType1Font(Standard14Fonts.FontName.HELVETICA_BOLD);
      PDType1Font regularFont = new PDType1Font(Standard14Fonts.FontName.HELVETICA);

      float y = 780;

      // Title Banner
      cs.beginText();
      cs.setFont(boldFont, 18);
      cs.newLineAtOffset(50, y);
      cs.showText("COLLEGE PLACEMENT MANAGEMENT SYSTEM");
      cs.endText();

      y -= 25;
      cs.beginText();
      cs.setFont(boldFont, 14);
      cs.newLineAtOffset(50, y);
      cs.showText("Official Executive Placement Report");
      cs.endText();

      y -= 15;
      cs.beginText();
      cs.setFont(regularFont, 10);
```

5.15 Database connectivity using JDBC

The application uses JDBC to connect the Java application layer to the MySQL database. Database connection settings are read from environment variables so that credentials and connection details are not hard-coded into the application source. DAO and service classes separate data access from request handling and business rules. This structure makes database operations easier to maintain and allows the application to reuse the same connection configuration across its modules.

dao/UserDAO.java

```
package com.cpms.dao;

public class UserDAO {

    public User findByEmail(String email) {
        if (email == null) return null;

        String sql = """
                SELECT user_id, full_name, email,
                       password_hash, role, is_active, created_at
                FROM users
                WHERE LOWER(email) = LOWER(?)
                """;

        try (Connection connection = DBConnection.getConnection();
             PreparedStatement statement = connection.prepareStatement(sql)) {

            statement.setString(1, email.trim());

            try (ResultSet resultSet = statement.executeQuery()) {
                if (resultSet.next()) {
                    return mapUser(resultSet);
                }
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return null;
    }

    public User findById(int userId) {
        String sql = """
                SELECT user_id, full_name, email,
                       password_hash, role, is_active, created_at
                FROM users
                WHERE user_id = ?
                """;

        try (Connection connection = DBConnection.getConnection();
             PreparedStatement statement = connection.prepareStatement(sql)) {

            statement.setInt(1, userId);

            try (ResultSet resultSet = statement.executeQuery()) {
                if (resultSet.next()) {
                    return mapUser(resultSet);
                }
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return null;
    }
```

5.16 Data validation and access control

Validation is applied at both the interface and service layers where appropriate. Required values are checked before records are stored, and role-based route protection is used to prevent unauthorized access to administrative or recruiter functions. Eligibility checks use the verified academic record and the requirements configured for the job. These controls improve consistency of stored information and ensure that the same business rules are applied when users access placement features.

filter/AuthFilter.java

```
package com.cpms.filter;

public class AuthFilter implements Filter {

   @Override
   public void doFilter(
         jakarta.servlet.ServletRequest request,
         jakarta.servlet.ServletResponse response,
         FilterChain chain)
         throws IOException, ServletException {

      HttpServletRequest req = (HttpServletRequest) request;
      HttpServletResponse res = (HttpServletResponse) response;

      String requestURI = req.getRequestURI();
      String contextPath = req.getContextPath();
      String path = requestURI.substring(contextPath.length());

      // Bypass public resources
      if (path.startsWith("/assets/") ||
         path.equals("/login.jsp") ||
         path.equals("/login") ||
         path.equals("/register.jsp") ||
         path.equals("/register") ||
         path.equals("/access-denied.jsp") ||
         path.equals("/index.jsp") ||
         path.equals("/") ||
         path.isEmpty()) {
         chain.doFilter(request, response);
         return;
      }

      HttpSession session = req.getSession(false);

      // Check login
      if (session == null || session.getAttribute("userId") == null) {
         res.sendRedirect(contextPath + "/login");
         return;
      }

      String role = (String) session.getAttribute("role");

      // Role-based access enforcement with friendly redirect
      if (path.startsWith("/student/") && !"STUDENT".equalsIgnoreCase(role)) {
         res.sendRedirect(contextPath + "/access-denied.jsp?required=STUDENT&current=" + (role != null ? role : "GUEST"));
         return;
      }

      if ((path.startsWith("/officer/") || path.startsWith("/admin/")) && !"OFFICER".equalsIgnoreCase(role)) {
         res.sendRedirect(contextPath + "/access-denied.jsp?required=OFFICER&current=" + (role != null ? role : "GUEST"));
         return;
      }

      if (path.startsWith("/recruiter/") && !"RECRUITER".equalsIgnoreCase(role)) {
         res.sendRedirect(contextPath + "/access-denied.jsp?required=RECRUITER&current=" + (role != null ? role : "GUEST"));
         return;
      }

      // Prevent caching of sensitive portal pages
      res.setHeader("Cache-Control", "no-cache, no-store, must-revalidate");
      res.setHeader("Pragma", "no-cache");
      res.setDateHeader("Expires", 0);

      chain.doFilter(request, response);
   }
}
```

util/PasswordUtil.java

```
package com.cpms.util;

public class PasswordUtil {
```

```
    public static String hashPassword(String password) {
        return BCrypt.hashpw(password, BCrypt.gensalt(12));
    }

    public static boolean checkPassword(
            String password, String hashedPassword) {
        return BCrypt.checkpw(password, hashedPassword);
    }
}
```

5.17 Testing and observed workflow

The completed application was reviewed through its principal user workflows: authentication, student profile maintenance, resume handling, job browsing, eligibility review, application tracking and role-specific dashboards. Administrative workflows include maintaining student, company and job records. Recruiter workflows include reviewing candidates and progressing recruitment activity. The supplied interface screenshots document these implemented screens. Final deployment should use the institution's approved database credentials and server configuration.

test/DatabaseConnectionTest.java

```
package com.cpms.test;

public class DatabaseConnectionTest {

    public static void main(String[] args) {

        try (Connection connection = DBConnection.getConnection()) {

            DatabaseMetaData metaData = connection.getMetaData();

            System.out.println("================================");
            System.out.println("DATABASE CONNECTION SUCCESSFUL");
            System.out.println("================================");

            System.out.println("Database: " +
                connection.getCatalog());

            System.out.println("MySQL Version: " +
                metaData.getDatabaseProductVersion());

            System.out.println("JDBC Driver: " +
                metaData.getDriverName());

        } catch (Exception e) {

            System.out.println("DATABASE CONNECTION FAILED");
            e.printStackTrace();
        }
    }
}
```

test/PasswordVerificationTest.java

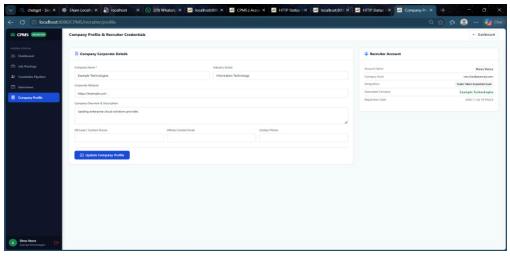
```
package com.cpms.test;

public class PasswordVerificationTest {

    public static void main(String[] args) {
        AuthService authService = new AuthService();

        System.out.println("--- TESTING ALL THREE LOGINS ---");
        String[][] tests = {
            {"officer@cpms.test", "Officer@123"},
            {"student@cpms.test", "Student@123"},
            {"recruiter@example.com", "Recruiter@123"}
        };

        for (String[] t : tests) {
            User u = authService.login(t[0], t[1]);
            System.out.println("Login " + t[0] + " with '" + t[1] + "' => " +
                (u != null ? ("SUCCESS [Name: " + u.getFullName() + ", Role: " + u.getRole() + "]") : "FAILED"));
        }
    }
}
```

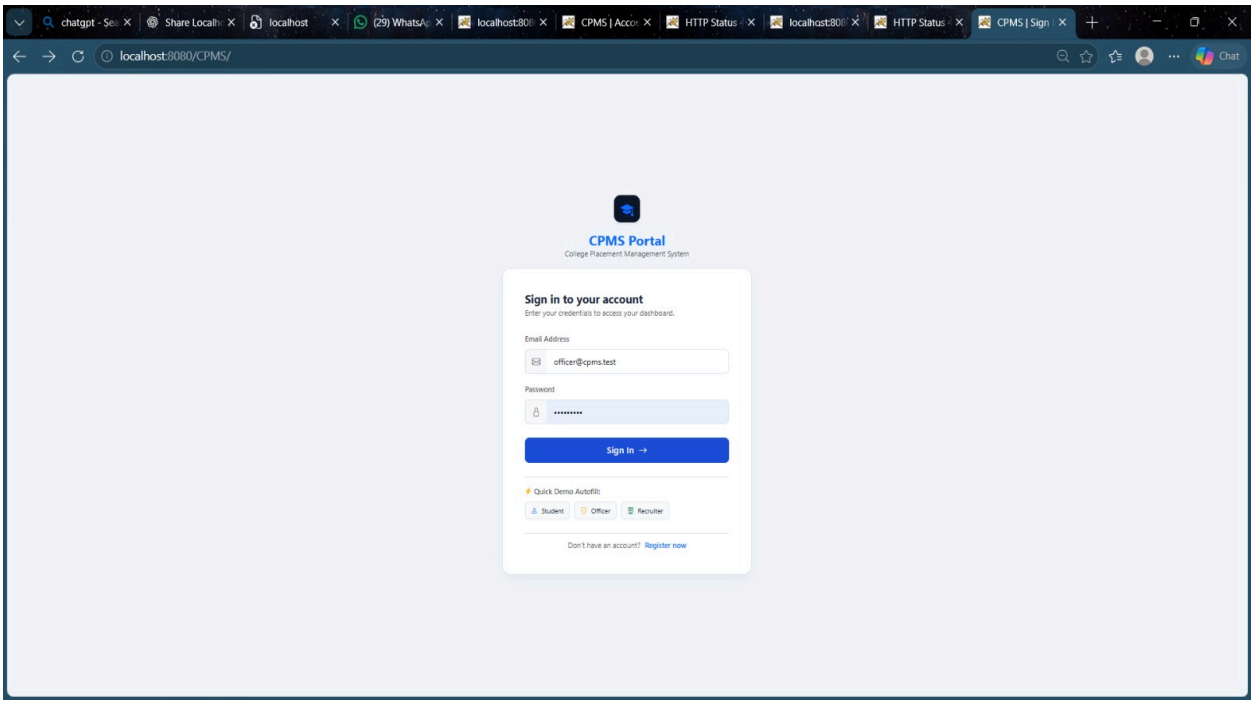


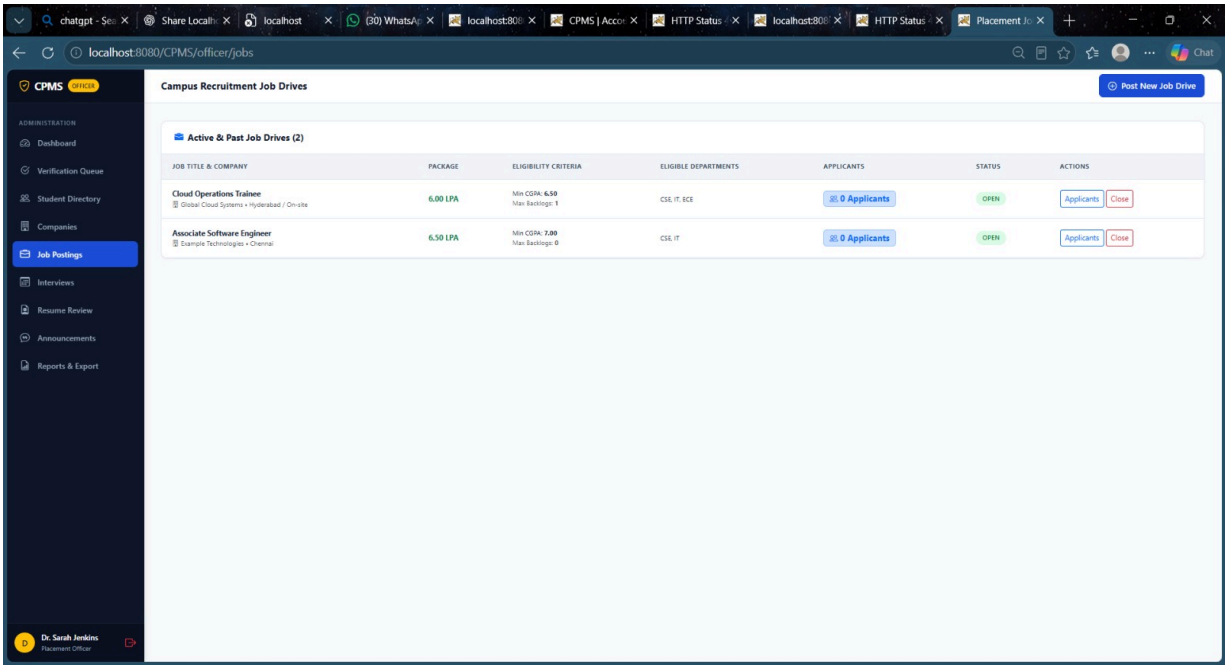
5.18 Implementation summary

The implementation combines presentation pages, servlet controllers, service logic, data-access classes and the MySQL schema. Each layer has a defined responsibility: pages present information, controllers handle requests, services apply application rules, DAO classes communicate with the database, and the schema stores persistent records. This separation makes the project easier to understand and extend. The implementation demonstrates the integration of Java programming concepts with relational database design in a practical placement-management context.

5.6 CLOSE OR DEACTIVATE RECORDS PAGE:

CLOSE OR DEACTIVATE RECORDS PAGE 37





dao/CompanyDAO.java (deactivate company)

```
public boolean toggleStatus(int companyId, boolean active) {
    String sql = "UPDATE companies SET is_active = ? WHERE company_id = ?";
    try (Connection con = DBConnection.getConnection();
         PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setBoolean(1, active);
        ps.setInt(2, companyId);
        return ps.executeUpdate() > 0;
    } catch (SQLException e) {
        e.printStackTrace();
        return false;
    }
```

service/OfficerService.java (close job)

```
}

public boolean closeJob(int jobId) {
    return jobDAO.setJobStatus(jobId, "CLOSED");
}

public List<Application> getJobApplicants(int jobId) {
    return applicationDAO.getApplicationsByJob(jobId);
}

public boolean updateApplicantStatus(int applicationId, String status, int officerId) {
    boolean updated = applicationDAO.updateStatus(applicationId, status);
    if (updated) {
        Application app = applicationDAO.findById(applicationId);
        if (app != null) {
            Student student = studentDAO.findById(app.getStudentId());
```

dao/ResumeDAO.java (deactivate old resume)

```
    try {
        con = DBConnection.getConnection();
        con.setAutoCommit(false);

        // Deactivate previous active resumes
        String deactSql = "UPDATE resumes SET is_active = FALSE WHERE student_id = ?";
        try (PreparedStatement psDeact = con.prepareStatement(deactSql)) {
            psDeact.setInt(1, studentId);
            psDeact.executeUpdate();
        }

        // Insert new active resume
        String insertSql = "INSERT INTO resumes (student_id, file_name, file_path, is_active) VALUES (?, ?, ?, TRUE)";
        int generatedId = -1;
        try (PreparedStatement psIns = con.prepareStatement(insertSql, Statement.RETURN_GENERATED_KEYS)) {
            psIns.setInt(1, studentId);
            psIns.setString(2, fileName);
            psIns.setString(3, filePath);
            psIns.executeUpdate();

            try (ResultSet rs = psIns.getGeneratedKeys()) {
                if (rs.next()) {
                    generatedId = rs.getInt(1);
                }
            }
        }
```

6.1 System benefits

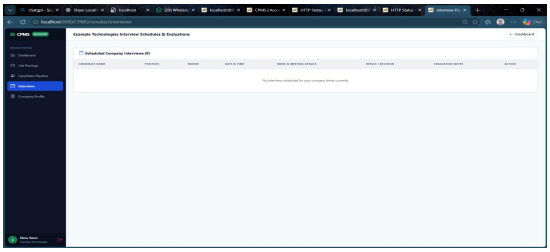
The system centralizes placement information that would otherwise be maintained in separate records. Students can access opportunities and track their applications, placement officers can coordinate company and job information, and recruiters can review candidates associated with their openings. Eligibility checking provides a consistent comparison between academic details and job requirements. Notifications and interview records support timely coordination. These functions make the placement workflow more visible to the participants.

6.2 Limitations and future enhancement

The system depends on accurate student data, current job criteria and reliable server and database configuration. File uploads and generated reports also require appropriate storage permissions and operational maintenance. Future work may include integration with institutional single sign-on, automated email delivery, richer analytics, calendar synchronization and improved audit reporting. Such enhancements should preserve the existing role boundaries and validate data before it is used for placement decisions.

6.3 Conclusion

The College Placement Management System demonstrates how Java Servlets, JSP, JDBC and MySQL can be combined to support a structured placement process. The project provides separate workspaces for students, placement officers and recruiters, while shared database records connect job opportunities, applications, interviews and notifications. The result is a practical academic project that applies object-oriented programming and database management concepts to a real institutional workflow.



6.4 Project outcome

The completed project provides a working foundation for managing the major stages of campus recruitment in one application. Its modular structure allows additional placement rules and reporting requirements to be introduced without redesigning every part of the system. With accurate institutional data and a properly configured deployment environment, CPMS can serve as a centralized platform for communicating opportunities and following recruitment progress.

6 CONCLUSION 42

IMPLEMENTATION 12

The project demonstrates how Java Servlets, JSP, JDBC and MySQL can be combined to build a practical placement management application. The use of role-based access, service classes and DAO-based database operations helps organize the application and supports consistent handling of student, company, job and application records.

Through this project, we gained practical experience in Java web application development, JDBC connectivity, relational database design and integration of multiple modules. CPMS provides a centralized interface for students, placement officers and recruiters to manage placement-related information, follow application progress, coordinate interviews and review placement outcomes. The system reduces dependence on scattered manual records and provides a structured foundation for future improvements.

7 REFERENCES 43

Oracle. Java Documentation and JDBC Documentation.

MySQL. MySQL 8.0 Reference Manual.

https://www.youtube.com/playlist?list=PLbKub4Jss9oUbTmeS9li_DoskYCiuvx1RApache Software Foundation. Apache Tomcat 10.1 Documentation; Apache POI Documentation; Apache PDFBox Documentation.